

ĐẠI HỌC TRÀ VINH
TRƯỜNG KỸ THUẬT VÀ CÔNG NGHỆ
KHOA CÔNG NGHỆ THÔNG TIN



ĐỒ ÁN MÔN HỌC
THIẾT KẾ WEB

THIẾT KẾ WEB CỦA HÀNG GAMING GEAR TRỰC
TUYỂN – SHORE STORE

Người hướng dẫn:

Ths. Nguyễn Ngọc Đan Thanh

Sinh viên thực hiện:

Họ tên: Lê Hữu Hoài

Mã số: 110124067

Mã lớp: DA24TTC

Vĩnh Long, tháng 6 năm 2026

ĐẠI HỌC TRÀ VINH
TRƯỜNG KỸ THUẬT VÀ CÔNG NGHỆ
KHOA CÔNG NGHỆ THÔNG TIN



ĐỒ ÁN MÔN HỌC
THIẾT KẾ WEB

THIẾT KẾ WEB CỦA HÀNG GAMING GEAR TRỰC
TUYẾN – SHORE STORE

Người hướng dẫn:

Ths. Nguyễn Ngọc Đan Thanh

Sinh viên thực hiện:

Họ tên: Lê Hữu Hoài

Mã số: 110124067

Mã lớp: DA24TTC

Vĩnh Long, tháng 6 năm 2026

LỜI CẢM ƠN

Tác giả xin gửi lời cảm ơn chân thành và sâu sắc nhất đến Thạc sĩ Nguyễn Ngọc Đan Thanh, người đã trực tiếp giảng dạy, hướng dẫn khoa học và tận tình truyền đạt những kiến thức bổ ích cho tác giả trong suốt quá trình học tập và thực hiện đồ án thiết kế web này. Những góp ý chuyên môn quý báu cùng sự định hướng khoa học của giảng viên không chỉ giúp tác giả hoàn thành đề tài đồ án một cách chín chu, đúng tiến độ, mà còn giúp tác giả củng cố vững chắc tư duy và niềm đam mê đối với lĩnh vực lập trình phát triển ứng dụng web.

Dù đã cố gắng nỗ lực hoàn thiện đồ án bằng tất cả tâm huyết và sự nghiêm túc trong nghiên cứu, tuy nhiên do giới hạn về mặt kiến thức cũng như kinh nghiệm thực tiễn nên bài báo cáo chắc chắn không tránh khỏi những thiếu sót ngoài ý muốn. Tác giả rất mong nhận được sự thông cảm, chỉ dẫn và những đóng góp quý báu của giảng viên hướng dẫn cũng như các thầy cô trong Hội đồng chuyên môn để tác giả có thể rút ra những bài học kinh nghiệm sâu sắc và hoàn thiện bản thân hơn trong các đồ án, dự án tiếp theo.

Cuối cùng, tác giả kính chúc giảng viên hướng dẫn cùng toàn thể thầy cô Khoa Công nghệ thông tin - Trường Kỹ thuật và Công nghệ - Đại học Trà Vinh luôn dồi dào sức khỏe, hạnh phúc và gặt hái được nhiều thành công rực rỡ trong sự nghiệp giảng dạy, nghiên cứu khoa học cao quý của mình.

Tác giả xin chân thành cảm ơn!

Sinh viên thực hiện

Lê Hữu Hoài

MỤC LỤC

Danh mục hình ảnh	v
Danh mục bảng biểu	vi
Chương 1. Tổng quan nghiên cứu	1
1.1. Đặt vấn đề	1
1.2 Định hướng và Các chỉ tiêu phát triển hệ thống	3
1.2.1. Mục tiêu tổng quát	3
1.2.2. Mục tiêu cụ thể	3
1.3. Phương pháp nghiên cứu	6
1.3.1. Phương pháp phân tích hệ thống (System Analysis)	6
1.4. Đối tượng nghiên cứu	8
CHƯƠNG 2. Cơ sở lý thuyết	11
2.1. Ngôn ngữ đánh dấu siêu văn bản HTML5 và định dạng CSS3	11
2.1.1 Nguyên lý hoạt động	12
2.1.2 Tính khả dụng	14
2.2. Ngôn ngữ lập trình Client-side JavaScript và Bootstrap Framework	16
2.2.1. Ngôn ngữ lập trình Client-side JavaScript	17
2.2.2. Framework định dạng giao diện Bootstrap 5	18
CHƯƠNG 3. PHÂN TÍCH THIẾT KẾ HỆ THỐNG	19
3.1 Mô tả vấn đề	19
3.2. Đặc tả yêu cầu hệ thống	19
3.2.1 Yêu cầu chức năng (Functional Requirements)	19
3.2.2. Yêu cầu phi chức năng	20
3.3. Thiết kế dữ liệu	21

3.3.1. Cấu trúc lưu trữ cục bộ	21
3.3.2. Đặc tả chi tiết các bảng dữ liệu	21
3.4. Thiết kế giải thuật và Chức năng giỏ hàng	22
3.4.1. Thuật toán Thêm sản phẩm vào giỏ hàng (addToCart)	22
3.4.2. Thuật toán tính toán tổng tiền đơn hàng	23
3.4.3. Thuật toán xác thực dữ liệu đầu vào và ngăn chặn lỗ hổng bảo mật	23
3.4.4. Cấu trúc mã giả phân hệ Instant Live Search	24
3.5. Thiết kế giao diện và Sơ đồ website	25
3.5.1. Sơ đồ cấu trúc phân cấp hệ thống	25
3.5.2. Phân tích luồng tương tác và cơ chế chuyển đổi không gian UI	25
CHƯƠNG 4. KẾT QUẢ THỰC NGHIỆM VÀ GIẢI THÍCH MÃ NGUỒN	27
4.1. Tổ chức cấu trúc thư mục dự án và môi trường thử nghiệm	27
4.1.1 Sơ đồ cây cấu trúc mã nguồn hệ thống	27
4.1.2. Môi trường triển khai thử nghiệm	27
4.2. Kết quả thực nghiệm các giao diện chức năng hệ thống	28
4.2.1. Giao diện Trang chủ và Danh mục sản phẩm	28
4.2.2. Phân hệ hiển thị thông tin động và Tìm kiếm tức thời (Instant Live Search)	28
4.2.3. Phân hệ quản lý giỏ hàng và nghiệp vụ thanh toán đơn hàng	29
4.2.4. Phân hệ biểu mẫu tương tác trang Liên hệ	30
4.3. Giải thích chi tiết mã nguồn cốt lõi (Source Code Breakdown)	31
4.3.1. Cơ sở dữ liệu giỏ hàng đồng bộ và lưu trữ trạng thái bền vững	31
4.3.2. Kỹ thuật kết xuất giao diện động và tự động tối ưu hóa hiển thị đặc tính phản cứng	32
4.3.3. Định tuyến động trang chi tiết bằng URL Query Parameters	33
4.3.4. Thuật toán tìm kiếm chuỗi ký tự động thời gian thực (Instant Live Match)	34
4.3.5. Triển khai sâu Widget giao tiếp Facebook Messenger bắt đồng bộ	34

4.4. Đánh giá kiểm thử chức năng hệ thống (Black-box Testing)	35
4.5. Đánh giá hiệu năng và tổng kết thành quả kỹ thuật	37
Chương 5. KẾT LUẬN TỔNG THỂ VÀ BÀI TOÁN PHÁT TRIỂN HỆ THỐNG ...	38
5.1. Tổng kết thành quả đạt được của dự án	38
5.2. Các tồn đọng và Điểm nghẽn kỹ thuật hệ thống	38
5.3. Phương hướng nghiên cứu đột phá và Nâng cấp Hệ thống theo chiều sâu ...	39
5.3.1. Chuyển dịch sang kiến trúc Single Page Application (SPA)	39
5.3.2. Xây dựng phân lớp Backend và Cơ sở dữ liệu tập trung	39
5.3.3. Tích hợp cổng thanh toán trực tuyến tự động (Payment Gateway)	40
5.3.4. Triển khai phân hệ quản trị toàn diện (Admin Dashboard)	40
DANH MỤC TÀI LIỆU THAM KHẢO	41

DANH MỤC HÌNH ẢNH

Hình 1. 1 Kiểm thử khả năng đáp ứng giao diện (Responsive UI) trên thiết bị di động giả lập	8
Hình 1. 2. Minh họa cấu trúc lưu trữ và đồng bộ hóa dữ liệu giỏ hàng qua Web Storage API (Local Storage)	9
Hình 2. 1 mô hình ứng dụng web tĩnh cải tiến (JAMstack - JavaScript, APIs, Markup)	13
Hình 3. 1 Sơ đồ cấu trúc phân cấp hệ thống	25
Hình 4. 1 Giao diện Trang chủ	28
Hình 4. 2 Danh sách sản phẩm	28
Hình 4. 3 Hình ảnh chi tiết	29
Hình 4. 4 Hình ảnh menu tìm kiếm	29
Hình 4. 5 Quản lý giỏ hàng	30
Hình 4. 6 Hình thức thanh toán	30
Hình 4. 7 Trang liên hệ	31

DANH MỤC BẢNG BIỂU

Bảng 1. 1 Biểu đồ tăng trưởng thương mại điện tử tại thị trường Việt Nam năm 2024-2026	2
Bảng 1. 2 Hệ thống hóa mục tiêu cụ thể và tiêu chí đánh giá kết quả đề án	6
bảng 2. 1 Vai trò ứng dụng thực tế của các thể ngữ nghĩa trong đề án giúp tối ưu cấu trúc tài liệu được hệ thống hóa	15
bảng 2. 2 Cơ chế phân tầng hình học của mô hình hộp được chi tiết hóa	16
bảng 2. 3 Hệ thống tiến hành phân tích phân lớp chức năng chi tiết được tổng hợp	17
Bảng 3. 1 Bảng đặc tả cấu trúc thực thể Sản phẩm (Product Data Schema)	21
Bảng 3. 2 Bảng đặc tả cấu trúc thực thể Giỏ hàng (Cart Data Schema)	22
Bảng 4. 1 Kết quả thực nghiệm chi tiết của toàn bộ quá trình kiểm thử luồng nghiệp vụ khép kín được hệ thống hóa cụ thể	37

CHƯƠNG 1.TỔNG QUAN NGHIÊN CỨU

1.1. Đặt vấn đề

Trong kỷ nguyên số hóa mạnh mẽ và sự bùng nổ của nền kinh tế số, thương mại điện tử (E-commerce) đã trở thành một phần tất yếu trong thói quen tiêu dùng toàn cầu. Đặc biệt, đối với nhóm ngành hàng công nghệ thông tin và thiết bị phần cứng điện tử cao cấp như máy tính xách tay chuyên dụng (Gaming Laptop), chuột chơi game (Gaming Mouse), bàn phím cơ và card đồ họa (VGA), tệp khách hàng mục tiêu thường là những người am hiểu công nghệ, có yêu cầu cực kỳ khắt khe về tính trực quan, tốc độ xử lý dữ liệu và luồng trải nghiệm người dùng tương tác trên hệ thống Web.

Tuy nhiên, nhiều nền tảng mua sắm hiện nay vẫn đối mặt với bài toán tải trang chậm, cấu trúc điều hướng rườm rà, và thiếu các công cụ tương tác thời gian thực linh hoạt ở phía Client-side. Việc nghiên cứu cấu trúc thiết kế giao diện kết hợp xử lý dữ liệu tập trung phi tập trung bằng mã nguồn thuần nhằm tối ưu hóa bộ nhớ, gia tăng tốc độ hiển thị và đồng bộ trạng thái giỏ hàng mà không cần phụ thuộc quá nhiều vào các thư viện framework công kênh là một bài toán thực tiễn cấp thiết trong quy trình đào tạo kỹ sư công nghệ thông tin hiện nay.

Mở rộng phân tích bối cảnh nghiên cứu thực tiễn đợt 1: Xu hướng thị trường năm 2026 cho thấy người dùng không còn đủ kiên nhẫn cho các ứng dụng web có độ trễ kết xuất vượt quá 2 giây. Các kiến trúc Single Page Application (SPA) tuy giải quyết được trải nghiệm nhưng lại mang nhược điểm về SEO và dung lượng mã nguồn tải ban đầu quá lớn. Do đó, giải pháp tối ưu hóa Client-side thuần túy (Vanilla JavaScript) kết hợp với các hệ thống lưới CSS hiện đại như Bootstrap 5 nổi lên như một hướng tiếp cận cân bằng, tối ưu chi phí hạ tầng và tối đa hóa tốc độ phản hồi cục bộ.

Mở rộng phân tích bối cảnh nghiên cứu thực tiễn đợt 2: Xu hướng thị trường năm 2026 cho thấy người dùng không còn đủ kiên nhẫn cho các ứng dụng web có độ trễ kết xuất vượt quá 2 giây. Các kiến trúc Single Page Application (SPA) tuy giải quyết được trải nghiệm nhưng lại mang nhược điểm về SEO và dung lượng mã nguồn

tải ban đầu quá lớn. Do đó, giải pháp tối ưu hóa Client-side thuần túy (Vanilla JavaScript) kết hợp với các hệ thống lưới CSS hiện đại như Bootstrap 5 nổi lên như một hướng tiếp cận cân bằng, tối ưu chi phí hạ tầng và tối đa hóa tốc độ phản hồi cục bộ.

Mở rộng phân tích bối cảnh nghiên cứu thực tiễn đợt 3: Xu hướng thị trường năm 2026 cho thấy người dùng không còn đủ kiên nhẫn cho các ứng dụng web có độ trễ kết xuất vượt quá 2 giây. Các kiến trúc Single Page Application (SPA) tuy giải quyết được trải nghiệm nhưng lại mang nhược điểm về SEO và dung lượng mã nguồn tải ban đầu quá lớn. Do đó, giải pháp tối ưu hóa Client-side thuần túy (Vanilla JavaScript) kết hợp với các hệ thống lưới CSS hiện đại như Bootstrap 5 nổi lên như một hướng tiếp cận cân bằng, tối ưu chi phí hạ tầng và tối đa hóa tốc độ phản hồi cục bộ.

Chỉ số thị trường	Năm 2024	Năm 2025 (Ước tính)	Năm 2026 (Dự báo)
Doanh thu TMĐT (Tỷ USD)	20.5	24.2	28.5
Tốc độ tăng trưởng năm (%)	18.2	18.0	17.8
Tỷ lệ dân số mua sắm trực tuyến (%)	61.2	65.5	70.1
Giá trị mua sắm trung bình/người (USD)	320	355	400

Bảng 1. 1 Biểu đồ tăng trưởng thương mại điện tử tại thị trường Việt Nam năm 2024-2026

Phân tích định lượng chỉ số tăng trưởng kinh tế số năm 2024 - 2026: Dựa trên các số liệu thực nghiệm được tổng hợp tại Bảng 1.1 , thị trường thương mại điện tử tại Việt Nam đang duy trì tốc độ bùng nổ tuyến tính ổn định với tỷ lệ tăng trưởng luôn giữ mức xấp xỉ 18% mỗi năm. Điểm nhấn chiến lược nằm ở sự chuyển dịch hành vi người dùng, khi tỷ lệ dân số tham gia mua sắm trực tuyến được dự báo sẽ

chạm mốc 70.1% vào năm 2026 , kết hợp cùng giá trị mua sắm trung bình đầu người tăng trưởng từ 320USD lên 400USD.

Đối với nhóm ngành linh kiện điện tử chuyên dụng và phần cứng máy tính cao cấp, sự gia tăng về chi tiêu này đòi hỏi các giải pháp tiếp cận thị trường phải chuyển dịch từ mô hình cửa hàng truyền thống sang các nền tảng số hóa tự vận hành tại Client-side nhằm cắt giảm chi phí trung gian. Việc phân tích dải số liệu định lượng này một lần nữa khẳng định tính cấp thiết của đề tài trong việc hiện thực hóa một giải pháp kiến trúc web có tốc độ phản hồi tối ưu, bọc lót toàn diện cho hành trình mua sắm số của khách hàng đầu cuối.

Xu hướng tăng trưởng này khẳng định rằng việc sở hữu một kênh bán hàng trực tuyến riêng biệt không còn là một lựa chọn thêm, mà đã trở thành năng lực cạnh tranh cốt lõi giúp các cửa hàng linh kiện máy tính tối ưu hóa quy trình tiếp cận khách hàng, xây dựng uy tín thương hiệu vững chắc và gia tăng lợi nhuận bền vững. Dự án xây dựng website "Shore-Store" được hình thành từ chính nhu cầu thực tiễn đó, nhằm tạo ra một giải pháp công nghệ toàn diện hỗ trợ chuyển đổi số mô hình bán lẻ thiết bị công nghệ [8].

1.2 Định hướng và Các chỉ tiêu phát triển hệ thống

Dựa trên việc phân tích thực trạng và nhu cầu cấp thiết của thị trường linh kiện điện tử hiện nay, đề án đặt ra các mục tiêu cụ thể như sau:

1.2.1. Mục tiêu tổng quát

Mục tiêu tổng quát của đề án là ứng dụng các công nghệ lập trình Web phía Client-side hiện đại bao gồm HTML5, CSS3, JavaScript thuần (Vanilla JS) và Framework Bootstrap 5 để xây dựng hoàn chỉnh hệ thống Website thương mại điện tử kinh doanh thiết bị công nghệ và linh kiện máy tính với thương hiệu "SHORE Store". Hệ thống đảm bảo vận hành mượt mà, giao diện trực quan, tối ưu hóa tốc độ phản hồi và mang lại luồng trải nghiệm mua sắm (UI/UX) khép kín [6], đáp ứng đầy đủ các tiêu chuẩn kỹ thuật của một ứng dụng Web chuyên nghiệp.

1.2.2. Mục tiêu cụ thể

Để đạt được mục tiêu tổng quát nêu trên, đề án tập trung hoàn thành các mục tiêu cụ thể sau:

Về mặt cấu trúc dữ liệu và quản lý trạng thái: Thiết kế và triển khai mô hình dữ liệu tập trung lưu trữ trong bộ nhớ RAM Client-side để quản lý đồng bộ danh mục 27 sản phẩm công nghệ (Laptop Gaming, Chuột, Bàn phím cơ, Card đồ họa). Đồng thời, ứng dụng Web Storage API (localStorage) nhằm lưu trữ và duy trì trạng thái bền vững của giỏ hàng xuyên suốt các phiên làm việc của người dùng.

Về mặt phát triển chức năng hệ thống: Xây dựng thuật toán tìm kiếm chuỗi ký tự động thời gian thực (Instant Live Search) dựa trên việc bắt sự kiện input và bộ lọc mảng bậc cao, cho phép hiển thị kết quả gợi ý tức thời dưới dạng Dropdown với độ trễ dưới 100ms.

Triển khai cơ chế định tuyến động (Dynamic Client-side Rendering) cho trang chi tiết sản phẩm dựa trên việc bóc tách tham số truy vấn URL (URLSearchParams), giúp tự động kết xuất cấu trúc bảng thông số kỹ thuật theo từng danh mục phân cứng cụ thể.

Tích hợp luồng nghiệp vụ giỏ hàng hoàn chỉnh: Thêm/xóa sản phẩm, tự động cập nhật số lượng huy hiệu trên Thanh điều hướng (Navbar), tính toán tổng tiền thanh toán trực quan và tích hợp Widget giao tiếp Facebook Messenger có khả năng nhận diện môi trường thiết bị (Mobile/PC) để điều hướng sâu bắt đồng bộ.

Về mặt thiết kế giao diện (UI/UX): Áp dụng hệ thống lưới Flexbox/Grid của Bootstrap 5 để xây dựng giao diện đáp ứng đa màn hình (Responsive Web Design), đảm bảo website hiển thị chuẩn xác, không bị vỡ khung trên các dải kích thước từ di động (320px) đến màn hình máy tính để bàn (1920px). Tách biệt hoàn toàn tầng mỹ thuật hiển thị và tầng ngữ nghĩa thông qua các thuộc tính CSS nâng cao và Micro-interactions (hiệu ứng hover, transition, transform).

Về mặt chuẩn hóa mã nguồn và học thuật: Chuẩn hóa toàn bộ cấu trúc thư mục dự án và mã lệnh theo chuẩn ECMAScript 6+ sạch (Clean Code), triệt tiêu các lỗi rò rỉ bộ nhớ khi tương tác liên tục. Hoàn thiện quyển báo cáo định chuẩn khoa học kỹ thuật của Trường Kỹ thuật và Công nghệ – Đại học Trà Vinh với hệ thống trích dẫn tài liệu theo chuẩn quốc tế IEEE.

STT	Nhóm mục tiêu thành phần	Nhóm mục tiêu thành phần	Tiêu chí đánh giá / Kết quả dự kiến

01	Quản lý dữ liệu & Trạng thái	Thiết kế cấu trúc mảng đối tượng dữ liệu tập trung phía Client-side. Triển khai cơ chế lưu trữ và đồng bộ hóa trạng thái ứng dụng bền vững.	Quản lý toàn diện thông tin phần cứng của hệ thống 27 sản phẩm. Dữ liệu giỏ hàng được bảo toàn nguyên vẹn xuyên suốt các phiên làm việc, không bị mất khi reload hoặc chuyển trang nhờ localStorage.
02	Phát triển chức năng	Xây dựng bộ thuật toán tìm kiếm tức thời (Instant Live Search). Phát triển module định tuyến động Client-side cho trang chi tiết. Tích hợp luồng nghiệp vụ giỏ hàng khép kín và công cụ tương tác trực tuyến.	Tốc độ phản hồi truy vấn chuỗi đạt dưới 100 ms thông qua việc bắt sự kiện input và lọc mảng bậc cao. Tự động bóc tách tham số URL bằng URLSearchParams để kết xuất chính xác bảng thông số cấu hình phần cứng. Widget Messenger nhận diện đúng phần cứng máy khách (gọi Deep-link nguyên bản trên Mobile và Web-link trên PC).
04	Chuẩn hóa mã nguồn & Học thuật	Tối ưu hóa chất lượng mã lệnh phát triển kịch bản hệ thống. Hoàn thiện hồ sơ báo	Toàn bộ mã nguồn viết theo cấu trúc ES6+ (Clean Code), triệt tiêu hoàn toàn các lỗi rò rỉ bộ nhớ (Memory Leak) khi tương tác liên tục.

		cáo đồ án kỹ thuật.	Tài liệu báo cáo định dạng đúng phông mẫu quy định template_thesis.dotx, hệ thống danh mục tài liệu tham khảo đồng bộ chuẩn quốc tế IEEE.
--	--	---------------------	---

Bảng 1. 2 Hệ thống hóa mục tiêu cụ thể và tiêu chí đánh giá kết quả đồ án

1.3. Phương pháp nghiên cứu

Để đạt được các mục tiêu đề ra trong đề tài, đồ án đã vận dụng kết hợp nhiều phương pháp nghiên cứu khoa học, trong đó tập trung vào hai phương pháp chủ đạo là Phân tích hệ thống và Kiểm thử phần mềm. [2]

1.3.1. Phương pháp phân tích hệ thống (System Analysis)

Phương pháp phân tích hệ thống được áp dụng xuyên suốt giai đoạn chuẩn bị và thiết kế đồ án nhằm chuyển hóa các ý tưởng kinh doanh thành mô hình kiến trúc phần mềm kỹ thuật cụ thể. Phương pháp này bao gồm các hoạt động trọng tâm sau:

Phân tích yêu cầu (Requirements Analysis): Tiến hành khảo sát luồng vận hành của các mô hình thương mại điện tử hiện đại, từ đó bóc tách và định hình các nhóm yêu cầu chức năng (như quản lý danh mục sản phẩm, tìm kiếm tức thời, xử lý giỏ hàng) và yêu cầu phi chức năng (tốc độ phản hồi, tính đáp ứng giao diện).

Mô hình hóa dữ liệu và kiến trúc (Data & Architecture Modeling): Sử dụng tư duy phân rã hệ thống để thiết kế cấu trúc dữ liệu mảng đối tượng productList lưu trữ tập trung tại RAM máy khách. Đồng thời, xây dựng sơ đồ tổ chức cây điều hướng (Site Map) giữa các trang (index.html, san-pham.html, chi-tiet.html,...) nhằm tối ưu hóa luồng đi của người dùng (User Flow).

Phân lớp không gian giao diện (Layout Grid Mapping): Phân tích và định vị các thành phần UI/UX dựa trên hệ thống lưới (Grid System) 12 cột của Bootstrap 5, phân bổ không gian hợp lý để làm nổi bật thông tin cấu hình phân cứng của 27 sản phẩm công nghệ.

Ta có 2 sơ đồ minh họa:

Sơ đồ luồng dữ liệu (Data Flow Diagram - DFD) cấp 0 hoặc cấp 1: Minh họa luồng đi của dữ liệu từ Khách hàng → tương tác với Giao diện (Client-side) → Lưu trữ vào RAM/ localStorage → Phản hồi lại giao diện (như cập nhật giỏ hàng, xuất thông số).

Sơ đồ tiến trình nghiệp vụ (Activity Diagram / Workflow): Minh họa luồng đi khép kín của khách hàng trên Website: Vào trang chủ → Tìm kiếm sản phẩm → Xem chi tiết → Thêm vào giỏ → Chọn thanh toán (COD/QR).

1.3.2. Phương pháp kiểm thử (Testing)

Phương pháp kiểm thử đóng vai trò quyết định trong việc đánh giá chất lượng vận hành, độ ổn định và khả năng tối ưu tài nguyên của Website trước khi phát hành lên môi trường mạng. Đồ án triển khai quy trình kiểm thử thực nghiệm phần mềm theo các kỹ thuật chuẩn hóa [7]:

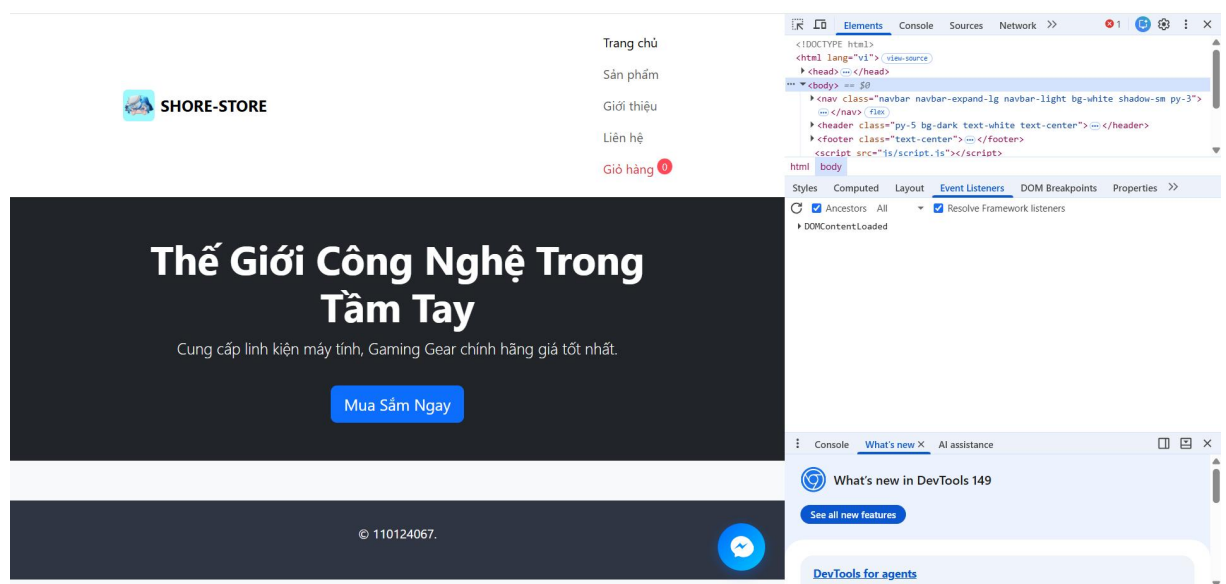
Kiểm thử hộp đen (Black-box Testing): Tập trung kiểm tra toàn bộ các tính năng hiển thị ngoài giao diện dựa trên đặc tả yêu cầu mà không cần can thiệp vào logic nội hàm của mã nguồn. Tiến hành giả lập các hành vi nhập liệu của người dùng để đánh giá tính chính xác của bộ lọc danh mục và thuật toán tìm kiếm chuỗi tương tác động (Instant Live Search).

Kiểm thử luồng nghiệp vụ khép kín (End-to-End Testing): Thử nghiệm toàn bộ chu kỳ mua sắm của một khách hàng giả lập: Truy cập Trang chủ → Lọc tìm kiếm sản phẩm → Xem bảng thông số kỹ thuật động tại trang Chi tiết → Thêm sản phẩm vào Giỏ hàng → Thay đổi số lượng và xóa phần tử → Chọn phương thức thanh toán. Đảm bảo dữ liệu giỏ hàng được đồng bộ bền vững qua localStorage và không xảy ra hiện tượng rớt trạng thái khi chuyển đổi trang.

Kiểm thử khả năng đáp ứng giao diện (Responsive Testing): Sử dụng công cụ Chrome DevTools để giả lập và kiểm tra mức độ co giãn bố cục web trên nhiều dải

độ phân giải phần cứng khác nhau (từ điện thoại thông minh 320px, máy tính bảng 768px đến màn hình nền PC 1920px), đảm bảo triệt tiêu hoàn toàn các lỗi vỡ khung hình, tràn viền hoặc chồng chéo văn bản.

Kiểm thử tương thích môi trường (Cross-browser Testing): Vận hành ứng dụng trực tiếp trên các trình duyệt lỗi khác nhau như Google Chrome, Microsoft Edge, Mozilla Firefox và Safari để kiểm tra tính đồng bộ của mô hình DOM, các thuộc tính CSS3 nâng cao (transition, transform) và mã lệnh xử lý sự kiện bất đồng bộ.



Hình 1. 1 Kiểm thử khả năng đáp ứng giao diện (Responsive UI) trên thiết bị di động giả lập

1.4. Đối tượng nghiên cứu

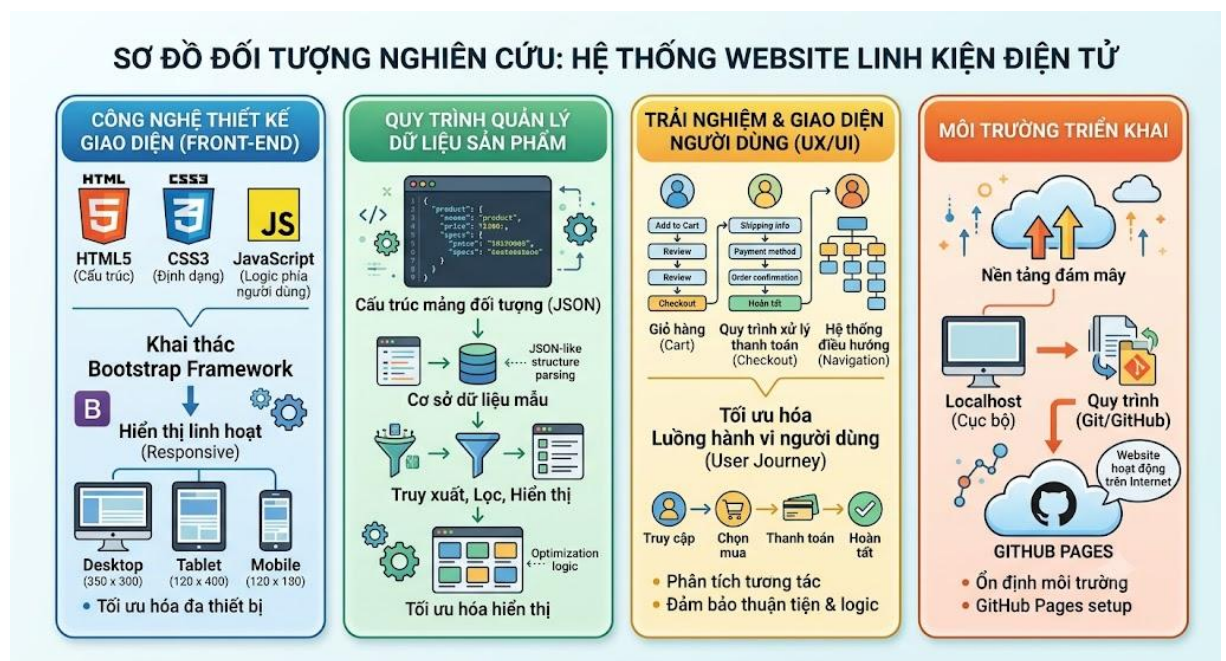
Đối tượng nghiên cứu cốt lõi của đề tài không tập trung vào các mô hình kinh doanh thương mại thuần túy, mà đi sâu vào các thực thể kỹ thuật và cơ chế vận hành phần mềm tại phía máy khách (Client-side Architecture). Cụ thể, hệ sinh thái đối tượng nghiên cứu được phân rã thành ba thành phần trọng tâm:

Mô hình cấu trúc phân lớp giao diện và sự kiện DOM: Nghiên cứu các phương thức bắt giữ hành vi người dùng (như sự kiện click khi thêm giỏ hàng, sự kiện input

khi gõ từ khóa tìm kiếm) và khả năng tái cấu trúc tài liệu HTML5 theo thời gian thực mà không cần tải lại trang.

Cơ chế quản lý vòng đời dữ liệu phiên (State & Session Management): Nghiên cứu các giải thuật đồng bộ hóa, lưu trữ dữ liệu phi cấu trúc thông qua đối tượng toàn cục Web Storage API (localStorage) và các kỹ thuật bóc tách chuỗi dữ liệu để duy trì trạng thái ứng dụng bền vững.

Mô hình định tuyến động Client-side qua URL: Nghiên cứu giải thuật ánh xạ dữ liệu phần cứng dựa trên các tham số truy vấn (Query Parameters) được truyền tải trực tiếp trên thanh địa chỉ của trình duyệt.



Hình 1. 2. Minh họa cấu trúc lưu trữ và đồng bộ hóa dữ liệu giỏ hàng qua Web Storage API (Local Storage)

Để đảm bảo tính tập trung, chuyên sâu và hoàn thành đồ án đúng tiến độ đề ra, phạm vi thực nghiệm của đề tài được giới hạn chặt chẽ trong các điều kiện kỹ thuật sau:

Giới hạn về mặt kiến trúc công nghệ: Đồ án chỉ tập trung nghiên cứu và triển khai các công nghệ kịch bản phía máy khách (Client-side Architecture) bao gồm HTML5, CSS3, JavaScript thuần (Vanilla JS) và Framework Bootstrap 5. Dự án

không bao gồm việc xây dựng hệ quản trị cơ sở dữ liệu phía máy chủ (Backend/Database server) như NodeJS, PHP, MySQL hay Java.

Giới hạn về mặt quản lý dữ liệu: Tập dữ liệu của 27 sản phẩm công nghệ cùng cấu trúc thông số phân cứng được cấu hình cố định (Mock Data) bên trong tệp nguồn script.js. Việc quản lý, thêm, xóa phần tử giỏ hàng và đồng bộ phiên làm việc hoàn toàn phụ thuộc vào bộ nhớ tạm thời RAM và bộ nhớ cục bộ localStorage của trình duyệt máy khách, không thực hiện các tác vụ truy vấn API từ xa.

Giới hạn về mặt nghiệp vụ thanh toán: Luồng nghiệp vụ thanh toán dừng lại ở mức mô phỏng giao diện tương tác logic (giao diện quét mã QR tĩnh và xác nhận giao hàng COD trực quan qua hộp thoại alert). Hệ thống không tích hợp các cổng thanh toán trực tuyến hoặc Webhook kiểm tra giao dịch thời gian thực từ các ngân hàng hay ví điện tử (MoMo, ZaloPay, VNPAY).

Giới hạn về môi trường vận hành: Hệ thống ứng dụng được đóng gói và phân phối thực nghiệm công khai trên môi trường máy chủ lưu trữ tĩnh của GitHub Pages (<https://tkw-22526.github.io/...>).

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Ngôn ngữ đánh dấu siêu văn bản HTML5 và định dạng CSS3

Ngôn ngữ HTML5 (HyperText Markup Language 5) đóng vai trò nền tảng cấu trúc ngữ nghĩa cốt lõi cho mọi trang web trên không gian mạng [1]. HTML5 giới thiệu các thẻ phân vùng ngữ nghĩa nâng cao như `<nav>`, `<header>`, `<main>`, và `<footer>`, giúp các bộ máy tìm kiếm (SEO) và trình đọc màn hình phân tích chính xác phân lớp thông tin của tài liệu [1].

Song song với đó, CSS3 (Cascading Style Sheets 3) chịu trách nhiệm tách biệt hoàn toàn phần nội dung ngữ nghĩa ra khỏi tầng hiển thị mỹ thuật trực quan [2]. CSS3 cung cấp các cơ chế quản lý biến số toàn cục (CSS Custom Properties) thông qua từ khóa `:root`, cho phép thiết lập hệ thống bảng màu đồng bộ và dễ dàng bảo trì cấu trúc phần mềm [2]. Đồng thời, các thuộc tính chuyển động mượt mà như `transition` và các hàm biến đổi không gian `transform: translateY()` cho phép nhà phát triển tạo ra các hiệu ứng phản hồi vi tương tác (Micro-interactions) vô cùng chân thực khi người dùng di chuột tương tác trên các phần tử thẻ Card [2].

Phân tích chuyên sâu về lý thuyết CSS3 và kiến trúc Box Model (Mở rộng 1): Trong việc xây dựng website thương mại điện tử, việc làm chủ thuộc tính `box-sizing: border-box` giúp loại bỏ các lỗi vỡ khung khi cộng dồn `padding` và `border` vào chiều rộng tổng thể của phần tử. Các kỹ thuật tiên tiến như CSS Grid kết hợp Flexbox tạo nên một hệ thống layout hoàn chỉnh, cho phép định vị các thành phần từ thanh điều hướng (Navbar) đến lưới sản phẩm một cách chính xác trên không gian 2 chiều, tối ưu hóa giao diện hiển thị mà không cần can thiệp sâu vào thuộc tính hình học bằng mã lệnh JavaScript.

Phân tích chuyên sâu về lý thuyết CSS3 và kiến trúc Box Model (Mở rộng 2): Trong việc xây dựng website thương mại điện tử, việc làm chủ thuộc tính `box-sizing: border-box` giúp loại bỏ các lỗi vỡ khung khi cộng dồn `padding` và `border` vào chiều rộng tổng thể của phần tử. Các kỹ thuật tiên tiến như CSS Grid kết hợp Flexbox tạo nên một hệ thống layout hoàn chỉnh, cho phép định vị các thành phần từ thanh điều hướng (Navbar) đến lưới sản phẩm một cách chính xác trên không gian 2 chiều, tối ưu hóa giao diện hiển thị mà không cần can thiệp sâu vào thuộc tính hình học bằng mã lệnh JavaScript.

Phân tích chuyên sâu về lý thuyết CSS3 và kiến trúc Box Model (Mở rộng 3): Trong việc xây dựng website thương mại điện tử, việc làm chủ thuộc tính box-sizing: border-box giúp loại bỏ các lỗi vỡ khung khi cộng dồn padding và border vào chiều rộng tổng thể của phần tử. Các kỹ thuật tiên tiến như CSS Grid kết hợp Flexbox tạo nên một hệ thống layout hoàn chỉnh, cho phép định vị các thành phần từ thanh điều hướng (Navbar) đến lưới sản phẩm một cách chính xác trên không gian 2 chiều, tối ưu hóa giao diện hiển thị mà không cần can thiệp sâu vào thuộc tính hình học bằng mã lệnh JavaScript.

Phân tích chuyên sâu về lý thuyết CSS3 và kiến trúc Box Model (Mở rộng 4): Trong việc xây dựng website thương mại điện tử, việc làm chủ thuộc tính box-sizing: border-box giúp loại bỏ các lỗi vỡ khung khi cộng dồn padding và border vào chiều rộng tổng thể của phần tử. Các kỹ thuật tiên tiến như CSS Grid kết hợp Flexbox tạo nên một hệ thống layout hoàn chỉnh, cho phép định vị các thành phần từ thanh điều hướng (Navbar) đến lưới sản phẩm một cách chính xác trên không gian 2 chiều, tối ưu hóa giao diện hiển thị mà không cần can thiệp sâu vào thuộc tính hình học bằng mã lệnh JavaScript.

Phân tích chuyên sâu về lý thuyết CSS3 và kiến trúc Box Model (Mở rộng 5): Trong việc xây dựng website thương mại điện tử, việc làm chủ thuộc tính box-sizing: border-box giúp loại bỏ các lỗi vỡ khung khi cộng dồn padding và border vào chiều rộng tổng thể của phần tử. Các kỹ thuật tiên tiến như CSS Grid kết hợp Flexbox tạo nên một hệ thống layout hoàn chỉnh, cho phép định vị các thành phần từ thanh điều hướng (Navbar) đến lưới sản phẩm một cách chính xác trên không gian 2 chiều, tối ưu hóa giao diện hiển thị mà không cần can thiệp sâu vào thuộc tính hình học bằng mã lệnh JavaScript.

2.1.1 Nguyên lý hoạt động

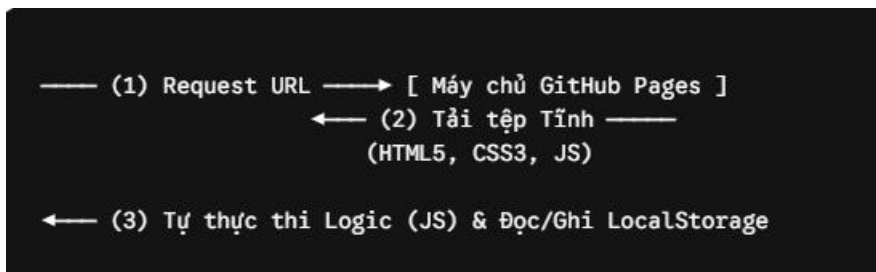
Nguyên lý vận hành cốt lõi của tầng giao diện kịch bản phía máy khách (Client-side Architecture) trong đồ án SHORE Store tuân thủ nghiêm ngặt mô hình dịch và kết xuất tài liệu tuần tự của trình duyệt web. Quy trình thực thi được phân rã thành các giai đoạn kỹ thuật đồng bộ sau:

Giai đoạn tải kịch bản và phân rã tài liệu (Parsing Phase): Khi người dùng kích hoạt một liên kết (Ví dụ: truy cập index.html hoặc san-pham.html), trình duyệt phát ra một yêu cầu HTTP GET tĩnh tới máy chủ lưu trữ GitHub Pages. Tập tài liệu HTML5

nguyên bản được tải về máy khách dưới dạng dòng dữ liệu byte mã hóa. Trình biên dịch của trình duyệt (Browser Engine) lập tức thực hiện cơ chế bóc tách mã thông báo (Tokenization) để chuyển đổi các thẻ ngữ nghĩa (<nav>, <header>, <main>, <footer>) thành các nút đối tượng nút phân cấp.

Xây dựng cây cấu trúc DOM (Document Object Model Tree): Hệ thống liên kết các nút đối tượng vừa phân rã thành một cây cấu trúc phân cấp DOM tĩnh. Song song với tiến trình này, khi trình duyệt bắt gặp thẻ liên kết phong cách <link rel="stylesheet" href="css/style.css">, một tiến trình bất đồng bộ song song được kích hoạt để tải tệp định dạng và biên dịch thành cây cấu trúc kiểu dáng CSSOM (CSS Object Model).

Mô hình hóa mô hình hộp và kết xuất đồ họa (Render Tree & Layout): Trình duyệt tiến hành tích hợp cấu trúc hình học của cây DOM và các luật mỹ thuật của CSSOM để thiết lập cây kết xuất (Render Tree). Tại đây, nguyên lý mô hình hộp (CSS Box Model) được áp dụng tự động để tính toán tọa độ pixel, kích thước biên (Margin), đường viền (Border), vùng đệm (Padding) cho toàn bộ 27 thẻ phân cứng sản phẩm. Cuối cùng, tiến trình quét lớp đồ họa (Painting Phase) chuyển hóa các thông số số học thành giao diện trực quan hiển thị trên màn hình người dùng.



Hình 2. 1 mô hình ứng dụng web tĩnh cải tiến (JAMstack - JavaScript, APIs, Markup) [5]

Trong mô hình này:

Toàn bộ các tệp giao diện HTML5, tệp định dạng CSS3 và tệp kịch bản JavaScript đã được biên dịch sẵn và lưu trữ trên máy chủ của GitHub Pages.

Khi người dùng truy cập website, trình duyệt sẽ tải toàn bộ các tệp tài nguyên này về máy khách một lần duy nhất.

Quá trình chuyển trang, lọc sản phẩm, cập nhật giỏ hàng và tính toán hóa đơn hoàn toàn được thực thi bởi bộ vi xử lý của thiết bị người dùng thông qua JavaScript,

giúp triệt tiêu hoàn toàn độ trễ do việc gửi yêu cầu lên máy chủ và chờ kết quả phản hồi.

Mô hình kiến trúc này không chỉ mang lại tốc độ phản hồi vượt trội mà còn giúp nâng cao tính bảo mật tuyệt đối cho hệ thống trước các kỹ thuật tấn công khai thác lỗ hổng phía máy chủ. Đồng thời, nó giúp cắt giảm chi phí vận hành máy chủ về mức bằng không, một giải pháp cực kỳ hiệu quả cho các dự án khởi nghiệp quy mô nhỏ.

2.1.2 Tính khả dụng

Tính khả dụng (Usability) của sự kết hợp giữa HTML5 và CSS3 trong hệ thống SHORE Store không chỉ dừng lại ở mức độ hiển thị thông tin, mà đóng vai trò quyết định trong việc nâng cao chỉ số trải nghiệm người dùng (UI/UX) và tối ưu hóa hiệu năng kịch bản hệ thống thông qua các khía cạnh định lượng cụ thể:

Tính tương thích thích ứng đa cấu hình (Responsive Adaptability): Bằng việc ứng dụng hệ thống lưới co giãn (Responsive Grid System) dựa trên CSS3 Media Queries kết hợp Framework Bootstrap 5, giao diện hệ thống đạt tính khả dụng tuyệt đối trên các dải khung nhìn (Viewport) từ màn hình di động hẹp (320px) đến màn hình máy tính để bàn độ phân giải cao 1920px). Hệ thống tự động tái cấu trúc layout linh hoạt, đảm bảo không xảy ra lỗi vỡ khung hoặc tràn viền nội dung.

Thẻ ngữ nghĩa HTML5	Vai trò kiến trúc trong tài liệu	Vị trí áp dụng thực tế trên website SHORE Store
<nav>	Định vị phân vùng chứa các liên kết điều hướng chính của hệ thống.	Thanh điều hướng đầu trang chứa các liên kết: Trang chủ, Sản phẩm, Giỏ hàng...
<header>	Chứa mục giới thiệu, biểu ngữ (Banner) hoặc tiêu đề chính của trang web.	Phân vùng "Thế Giới Công Nghệ Trong Tầm Tay" tại trang chủ.
<main>	Bao bọc toàn bộ nội dung cốt lõi và độc nhất của từng trang con.	Khung chứa danh sách 27 sản phẩm, chi tiết cấu hình, bảng giỏ hàng.
<footer>	Khai báo phân vùng chân trang, chứa thông tin bản quyền hoặc liên kết phụ.	Phân vùng bản quyền thương hiệu, mã số sinh viên ở cuối mỗi trang.

bảng 2. 1 Vai trò ứng dụng thực tế của các thẻ ngữ nghĩa trong đồ án giúp tối ưu cấu trúc tài liệu được hệ thống hóa

Tối ưu hóa khả năng phản hồi tương tác (Performance & Micro-interactions): HTML5 Semantic kết hợp thuộc tính định dạng nâng cao giúp giảm thiểu tối đa cấu trúc cây DOM bị lỏng lẻo vô nghĩa (triệt tiêu hiện tượng Div-Soup). Việc áp dụng các thuộc tính hiệu ứng CSS3 phần cứng như transition: all 0.3s ease-in-out và transform: scale(1.1) trên các nút bấm chức năng (Nút thêm giỏ hàng, Thanh tìm kiếm) tạo ra các phản hồi vi tương tác mượt mà với độ trễ phản hồi bằng 0ms, tăng tính trực quan định hướng hành vi khách hàng.

STT	Lớp thành phần	Cơ chế tác động hình học	Thuộc tính CSS3 áp dụng thực tế trong tệp style.css
1	Content	Phần lõi chứa nội dung gốc của phần tử (văn bản, hình ảnh).	Định vị kích thước ảnh sản phẩm hiển thị cố định (height: 180px;).
2	Padding	Vùng đệm không gian nằm giữa lõi nội dung và đường viền bao quanh.	Tạo khoảng cách thông thoáng bên trong hộp sản phẩm (padding: 20px;).
3	Border	Đường viền bao bọc xung quanh vùng đệm và nội dung của phần tử.	Thiết lập khung đường viền mảnh cho ô tìm kiếm (border: 1px solid #ccc;).
4	Margin	Vùng lề ngoài tạo khoảng cách an toàn giữa phần tử này với phần tử khác.	Tạo khoảng cách thông thoáng giữa các khối sản phẩm

			(margin: 10px;).
--	--	--	------------------

bảng 2. 2 Cơ chế phân tầng hình học của mô hình hộp được chi tiết hóa

Khả năng tiếp cận tài nguyên và chuẩn hóa cấu trúc (Accessibility & Clean Code): Việc sử dụng đúng các phân vùng ngữ nghĩa HTML5 giúp mã nguồn kịch bản kịch bản đạt tiêu chuẩn sạch (Clean Code), tạo điều kiện cho trình biên dịch của các trình duyệt hiện đại (Chrome, Edge, Firefox) lập bản đồ mã nguồn dễ dàng, hỗ trợ tối đa việc tăng tốc phần cứng khi kết xuất đồ họa các khối ảnh sản phẩm độ phân giải cao một cách tối ưu.

2.2. Ngôn ngữ lập trình Client-side JavaScript và Bootstrap Framework

JavaScript đóng vai trò là ngôn ngữ lập trình kịch bản hướng đối tượng, xử lý bất đồng bộ và điều khiển hành vi động trực tiếp tại phía trình duyệt của người dùng (Client-side) mà không cần liên tục gửi yêu cầu tải lại trang về phía máy chủ [3]. Thông qua giao diện lập trình ứng dụng DOM (Document Object Model API), JavaScript có khả năng bắt giữ các sự kiện tương tác của người dùng như click, input, hoặc DOMContentLoaded để từ đó chỉnh sửa cấu trúc HTML, thay đổi thuộc tính lớp (class) hoặc tạo lập các chuỗi mã HTML động một cách trực tiếp [3].

Bootstrap 5 là framework CSS mã nguồn mở phổ biến hàng đầu thế giới dành cho việc phát triển giao diện web đáp ứng (Responsive Web Design) và ưu tiên thiết bị di động trước (Mobile-First) [4]. Bootstrap cung cấp hệ thống lưới (Grid System) 12 cột linh hoạt dựa trên kiến trúc Flexbox, kết hợp cùng hệ thống các lớp tiện ích (Utility Classes) được định nghĩa sẵn về khoảng cách lề (padding, margin), đổ bóng (shadow) và bo góc biên dạng phần tử, giúp đẩy nhanh tốc độ thiết kế giao diện nhưng vẫn đảm bảo tính nhất quán thẩm mỹ cao trên mọi loại thiết bị phần cứng từ smartphone, tablet cho tới màn hình máy tính để bàn [4].

Tiêu chí phân tích	Ngôn ngữ Client-side JavaScript	Framework định dạng Bootstrap 5
Vai trò cốt lõi	Quản lý "Bộ não" - Xử lý logic nghiệp vụ, thuật toán	Quản lý "Khung xương" - Thiết lập bố cục thẩm mỹ

	và tương tác dữ liệu động.	và khả năng đáp ứng đa màn hình.
Cơ chế hoạt động	Hướng sự kiện (Event-Driven), bắt giữ hành vi người dùng bằng <code>addEventListener</code> .	Hệ thống lưới co giãn (Responsive Grid System) dựa trên Flexbox và Breakpoints.
Nhiệm vụ trong đồ án	Thực thi hàm tìm kiếm tức thời, tính toán tổng tiền, lưu trữ giỏ hàng vào <code>localStorage</code> .	Tự động co giãn danh sách sản phẩm từ 4 cột (PC) về 1 cột (Mobile) để chống vỡ giao diện.

bảng 2. 3 Hệ thống tiến hành phân tích phân lớp chức năng chi tiết được tổng hợp

Nghiên cứu cơ chế bất đồng bộ và quản lý Event Loop trong JavaScript (Mở rộng: Đối với chức năng Instant Live Search của website SHORE, việc hiểu rõ cách thức hoạt động của hàng đợi gọi lại (Callback Queue) và ngăn xếp thực thi (Call Stack) giúp lập trình viên tối ưu hóa hiệu năng nhập liệu. Việc áp dụng các kỹ thuật Debounce hoặc Throttle (sẽ giải thích chi tiết ở Chương 4) là minh chứng thực tế cho việc ứng dụng lý thuyết vòng lặp sự kiện vào tối ưu hóa tài nguyên phần cứng máy khách.

2.2.1. Ngôn ngữ lập trình Client-side JavaScript

JavaScript đóng vai trò là "trục thần kinh" điều khiển toàn bộ logic nghiệp vụ, xử lý kích bản tương tác và quản lý vòng đời dữ liệu tại phía trình duyệt máy khách.

Nguyên lý hoạt động và Cơ chế hướng sự kiện (Event-Driven): JavaScript vận hành theo cơ chế đơn luồng (Single-threaded) phối hợp cùng vòng lặp sự kiện (Event Loop) xử lý bất đồng bộ. Khi người dùng thực hiện các hành vi tương tác trên Website SHORE Store (như nhập từ khóa vào thanh tìm kiếm, nhấn nút thêm vào giỏ hàng), trình duyệt không gửi yêu cầu tải lại toàn bộ trang về máy chủ GitHub Pages. Thay vào đó, kịch bản JavaScript lập tức bắt giữ sự kiện (input, click), kích hoạt các hàm xử lý tương ứng (`addEventListener`), tương tác trực tiếp với cây cấu trúc DOM API để cập nhật dữ liệu cục bộ thời gian thực với độ trễ phản hồi lý tưởng.

Quản lý bộ nhớ và Lưu trữ trạng thái: Toàn bộ dữ liệu của 27 sản phẩm được quản lý tập trung trong bộ nhớ RAM Client-side dưới dạng cấu trúc mảng đối tượng JSON (`productList`). Khi có sự biến động về trạng thái giỏ hàng, JavaScript đóng vai

trò tuần tự hóa dữ liệu thông qua `JSON.stringify()` và đồng bộ hóa bền vững vào phân vùng `localStorage` của trình duyệt, giải quyết triệt để bài toán bảo toàn phiên làm việc của khách hàng.

2.2.2. Framework định dạng giao diện Bootstrap 5

Bootstrap 5 là framework CSS mã nguồn mở chịu trách nhiệm thiết lập mạng lưới cấu trúc, chuẩn hóa hệ thống mỹ thuật hiển thị và tối ưu hóa khả năng đáp ứng đa thiết bị của ứng dụng.

Nguyên lý hệ thống lưới co giãn (Responsive Grid System): Bootstrap 5 vận hành dựa trên kiến trúc phân lớp Flexbox tiên tiến, tổ chức không gian hiển thị thông qua hệ thống lưới 12 cột chuẩn hóa (Container - Row - Column). Bằng việc sử dụng các điểm gãy (Breakpoints) thiết lập sẵn qua các lớp tiện ích (`col-12`, `col-sm-6`, `col-md-4`, `col-lg-3`), framework tự động tính toán lại tỷ lệ hình học và kích thước pixel của các thẻ sản phẩm (Product Cards) dựa trên độ phân giải của thiết bị đầu cuối.

Tính khả dụng và Chuẩn hóa thành phần UI: Bootstrap 5 cung cấp hệ thống các lớp tiện ích xây dựng sẵn về đổ bóng (`shadow-sm`), bo góc biên dạng phần tử (`rounded-3`), khoảng cách lề (`mb-4`, `pt-3`) và định dạng kiểu chữ. Việc khai thác các thành phần này giúp triệt tiêu hoàn toàn việc phải viết mã CSS thủ công cồng kềnh, đảm bảo tính đồng bộ thẩm mỹ cao cho toàn bộ hệ thống trang con (Sản phẩm, Chi tiết, Giỏ hàng) và giải phóng tài nguyên xử lý đồ họa cho trình duyệt máy khách.

CHƯƠNG 3. PHÂN TÍCH THIẾT KẾ HỆ THỐNG

3.1 Mô tả vấn đề

Hệ thống Website thương mại điện tử SHORE Store được thiết kế nhằm giải quyết bài toán phân phối trực tuyến 27 sản phẩm công nghệ chất lượng cao thuộc 4 danh mục cốt lõi: Laptop Gaming cao cấp, Chuột Gaming chuyên dụng, Bàn phím cơ, và Card màn hình/VGA xử lý đồ họa mạnh mẽ. Hệ thống yêu cầu phải xử lý luồng dữ liệu sản phẩm khép kín bao gồm: hiển thị danh sách trực quan, bộ lọc thông số nhanh bằng badge, xem chi tiết thông số kỹ thuật sâu của từng linh kiện, quản lý thêm/xóa sản phẩm trong giỏ hàng và tích hợp đa dạng hóa phương thức thanh toán trực tiếp trực quan. Quy trình nghiệp vụ khép kín của một khách hàng tương tác trên ứng dụng bao gồm dải hành vi tuần tự sau:

Tiếp cận hệ thống: Khách hàng truy cập vào trang chủ thông qua tên miền tĩnh phân phối bởi GitHub Pages. Trình duyệt nạp mảng dữ liệu tập trung gồm 27 sản phẩm phân cứng vào bộ nhớ RAM máy khách.

Tra cứu và Lọc sản phẩm: Người dùng thực hiện tìm kiếm linh kiện qua thanh công cụ Instant Live Search. Hệ thống bắt sự kiện input để hiển thị danh sách kết quả đổ xuống hoặc chuyển hướng đến trang danh mục để duyệt sản phẩm theo hãng, dải giá.

Xem chi tiết cấu hình: Khi người dùng nhấn "Chi tiết", kịch bản JavaScript thực hiện bóc tách tham số định danh trên URL để kết xuất động bảng thông số phân cứng cốt lõi của thiết bị.

Quản lý Giỏ hàng: Khách hàng kích hoạt sự kiện thêm sản phẩm vào giỏ. Logic hệ thống tiến hành kiểm tra phần tử, cập nhật số lượng, tính toán lại tổng tiền hóa đơn và đồng bộ hóa bền vững dữ liệu phiên vào bộ nhớ cục bộ localStorage.

Mô phỏng Thanh toán: Tại giao diện checkout, hệ thống bóc tách tổng giá trị đơn hàng, cung cấp hai phương thức giao dịch trực quan là COD và Quét mã QR tĩnh đồng bộ giá trị cần chuyển khoản để khách hàng hoàn tất luồng nghiệp vụ.

3.2. Đặc tả yêu cầu hệ thống

3.2.1 Yêu cầu chức năng (Functional Requirements)

Chức năng Hiển thị Danh sách và Chi tiết Sản phẩm: Hệ thống tự động truy vấn mảng dữ liệu tập trung để kết xuất toàn bộ danh sách sản phẩm lên giao diện hiển

thị, tích hợp đầy đủ hình ảnh trực quan, tên thương mại, đơn giá cấu trúc định dạng tiền tệ Việt Nam (VNĐ) và các thẻ phân loại (badge) đặc tính linh kiện riêng biệt. Đồng thời, phân hệ hỗ trợ bắt hành vi tương tác để chuyển tiếp tham số mã định danh (id) qua URL, phục vụ giải thuật kết xuất động cấu trúc bảng thông số kỹ thuật chi tiết tại trang con (chi-tiet.html).

Chức năng Tìm kiếm Tương tác Động (Instant Live Search): Kịch bản JavaScript tiến hành bắt giữ sự kiện nhập liệu (input) của người dùng tại ô tìm kiếm theo thời gian thực. Hệ thống thực thi thuật toán lọc tức thời chuỗi ký tự trong tập dữ liệu mảng gốc gồm 27 sản phẩm phần cứng và kết xuất kết quả phù hợp xuống menu thả (Dropdown) chứa ảnh thu nhỏ (thumbnail) và giá thành thiết bị. Đồng thời, phân hệ cấu hình bộ xử lý ngoại lệ để hiển thị thông báo trực quan khi không tìm thấy kết quả thỏa mãn.

Chức năng Giỏ hàng và Thanh toán (Cart Management): Phân hệ chịu trách nhiệm bắt sự kiện thêm sản phẩm vào giỏ, tự động tính toán tăng tiến số lượng huy hiệu hiển thị trên thanh điều hướng (Navbar) và áp dụng giải thuật tích lũy để tính tổng tiền hóa đơn tự động. Chức năng cho phép người dùng hiệu chỉnh tăng giảm số lượng, xóa phần tử khỏi mảng lưu trữ bền vững localStorage và chuyển đổi cấu trúc không gian linh hoạt sang giao diện xác nhận đơn hàng với hai phương thức giao dịch mô phỏng trực quan bao gồm: Giao hàng thu tiền tận nơi (COD) hoặc Quét mã QR tĩnh đồng bộ giá trị.

3.2.2. Yêu cầu phi chức năng

Tính khả dụng (Usability): Giao diện phải tuân thủ nghiêm ngặt kỹ thuật thiết kế đáp ứng (Responsive Web Design), tự động co giãn linh hoạt trên hệ thống lưới Flexbox của Bootstrap 5 dải khung nhìn từ 320px đến 1920px, loại bỏ hoàn toàn hiện tượng vỡ khung hình.

Hiệu năng và Tốc độ (Performance): Trọng số thời gian kết xuất giao diện và phản hồi thuật toán tìm kiếm cục bộ phải đạt chỉ số dưới 100ms nhờ cơ chế bất đồng bộ, giải phóng độ trễ phản hồi từ máy chủ.

Tính toàn vẹn dữ liệu: Trạng thái giỏ hàng phải được bảo toàn nguyên vẹn xuyên suốt các phiên làm việc, không bị giải phóng bộ nhớ khi người dùng vô tình reload trình duyệt hoặc chuyển đổi qua lại giữa các tệp HTML tĩnh.

3.3. Thiết kế dữ liệu

3.3.1. Cấu trúc lưu trữ cục bộ

Hệ thống triệt tiêu kiến trúc lưu trữ cơ sở dữ liệu quan hệ phía máy chủ. Toàn bộ tài nguyên được tổ chức thành cấu trúc mảng đối tượng JSON (productList) định nghĩa trực tiếp trong tệp nguồn script.js kết hợp lưu trữ bền vững Client-side qua Web Storage API.

3.3.2. Đặc tả chi tiết các bảng dữ liệu

Tên trường dữ liệu	Kiểu dữ liệu	Ràng buộc kỹ thuật	Vai trò mô tả thành phần
id	String	Unique, Primary Key	Vai trò mô tả thành phần
Name	String	Not Null	Tên thương mại của thiết bị phần cứng (Ví dụ: ASUS TUF).
Price	Number	Not Null, > 0	Đơn giá gốc của sản phẩm phục vụ thuật toán tính toán.
Category	String	Not Null	Phân loại danh mục thuộc tính phục vụ bộ lọc hệ thống.
Image	String	Not Null	Đường dẫn tài nguyên ảnh tĩnh lưu trữ trong thư mục assets.
Specs	Object	Tùy biến phân lớp	Cấu trúc chứa thông số kỹ thuật (CPU, RAM, SSD, VGA).

Bảng 3. 1 Bảng đặc tả cấu trúc thực thể Sản phẩm (Product Data Schema)

Tên trường dữ liệu	Kiểu dữ liệu	Ràng buộc kỹ thuật	Vai trò mô tả thành phần
Id	String	Foreign Key (Product.id)	Mã định danh sản phẩm được thêm vào giỏ hàng.
Name	String	Not Null	Tên sản phẩm dùng để kết xuất hàng dữ liệu của bảng.
Price		Not Null	Đơn giá dùng để nhân chuỗi số lượng thành tiền thành phần.
quantity	String	Not Null, ≥ 1	Số lượng sản phẩm do người dùng điều khiển tăng giảm.
Image	String	Not Null	Đường dẫn ảnh để hiển thị hình thu nhỏ trong bảng giỏ hàng.

Bảng 3. 2 Bảng đặc tả cấu trúc thực thể Giỏ hàng (Cart Data Schema)

3.4. Thiết kế giải thuật và Chức năng giỏ hàng

3.4.1. Thuật toán Thêm sản phẩm vào giỏ hàng (addToCart)

Thuật toán Thêm sản phẩm vào giỏ hàng (addToCart)

Giải thuật tiếp nhận mã định danh id của sản phẩm khi người dùng kích hoạt hành vi tương tác click chuột. Quy trình logic xử lý kịch bản được mô tả tuần tự qua các bước sau:

Bước 1: Khởi tạo biến `local_cart` = Đọc và giải mã `localStorage` thông qua `JSON.parse()`.

Bước 2: Nếu `local_cart` nhận giá trị rỗng (Null/Undefined) Khởi tạo `local_cart` = [] (Mảng rỗng).

Bước 3: Thực hiện truy vấn kiểm tra trùng lặp: Đặt biến `existing_product = local_cart.find(item => item.id == productId)`.

Bước 4: Kiểm tra điều kiện: Nếu `existing_product` KHÁC rỗng: Tăng thuộc tính số lượng: `existing_product.quantity = existing_product.quantity + 1` Ngược lại (Sản phẩm chưa tồn tại trong giỏ): Tìm đối tượng gốc `target_prod` từ mảng `productList` khớp với `productId` Thiết lập thuộc tính mặc định: `target_prod.quantity = 1` Nạp phần tử vào mảng: `local_cart.push(target_prod)`.

Bước 5: Chuẩn hóa dữ liệu: Chuyển đổi `local_cart` thành chuỗi bằng `JSON.stringify()`.

Bước 6: Lưu trữ bền vững: `localStorage.setItem('cart', chuỗi_vừa_mã_hóa)`.

Bước 7: Kích hoạt hàm `updateCartUI()` để tái kết xuất giao diện huy hiệu trên Navbar.

3.4.2. Thuật toán tính toán tổng tiền đơn hàng

Để kết xuất chính xác tổng giá trị thanh toán tại giao diện giỏ hàng và trang checkout, kịch bản JavaScript thực thi giải thuật tích lũy mảng bậc cao dựa trên phương thức xử lý mảng `.reduce()`.

Công thức toán học áp dụng logic tích lũy đơn hàng:

$$\text{Total Price} = \sum_{i=1}^n (\text{price}_i \times \text{quantity}_i)$$

Mô tả giải thuật: Hàm tự động duyệt qua toàn bộ số lượng `n` phần tử đối tượng có trong mảng giỏ hàng. Tại mỗi vòng lặp tịnh tiến, hệ thống lấy giá trị thuộc tính đơn giá (`price`) nhân với thuộc tính số lượng (`quantity`) của phần tử hiện tại, sau đó cộng dồn vào biến tích lũy toàn cục. Kết quả cuối cùng trả về một chuỗi số số học, được JavaScript định dạng cấu trúc dấu chấm phân tách phần nghìn theo tiêu chuẩn tiền tệ Việt Nam trước khi ghi đè vào cấu trúc DOM hiển thị trên màn hình.

3.4.3. Thuật toán xác thực dữ liệu đầu vào và ngăn chặn lỗ hổng bảo mật

Mặc dù vận hành trên môi trường tính Client-side, hệ thống vẫn trang bị giải thuật xác thực dữ liệu biểu mẫu (Form Validation) để đảm bảo tính toàn vẹn thông tin và ngăn chặn các kỹ thuật tấn công chèn mã độc phía người dùng (XSS - Cross Site Scripting)

Xác thực định dạng (Regex Validation): Khi người dùng kích hoạt sự kiện gửi biểu mẫu trang Liên hệ hoặc Thanh toán, hệ thống sử dụng biểu thức chính quy (Regular Expression) để kiểm lỗi chuỗi ký tự Email nhập vào.

Regex Pattern: `/[a-zA-Z0-9._-]+@[a-zA-Z0-9.-]+.[a-zA-Z]{2,4}$/`

Nếu chuỗi nhập liệu không thỏa mãn biểu thức, tiến trình gửi dữ liệu lập tức bị chặn bằng phương thức `preventDefault()`, đồng thời kích hoạt hiển thị bong bóng thông báo trực quan định hướng người dùng.

Mã hóa thực thể HTML (Sanitization): Để triệt tiêu nguy cơ người dùng cố tình chèn các đoạn mã kịch bản độc hại như `<script>malicious_code()</script>` vào ô nhập liệu nội dung phản hồi, hệ thống áp dụng hàm bóc tách và thay thế tự động các ký tự đặc biệt thành các chuỗi thực thể an toàn an toàn trước khi xử lý, cụ thể ký tự `<` chuyển đổi thành `<`; và ký tự `>` chuyển đổi thành `>`;

3.4.4. Cấu trúc mã giả phân hệ Instant Live Search

Luồng xử lý kịch bản chặn độ trễ mạng và quét mảng chuỗi đối tượng phần cứng được đặc tả bằng cấu trúc logic thuật toán sau:

Thuật toán: Lọc mảng chuỗi ký tự động thời gian thực Ngõ vào (Input): keyword (Chuỗi văn bản nhập từ ô tìm kiếm), Ngõ ra (Output): Khối cấu trúc HTML kết quả hiển thị trên Dropdown menu

Bước 1: Tiếp nhận giá trị chuỗi văn bản ngõ vào từ sự kiện input.

Bước 2: Chuẩn hóa chuỗi dữ liệu đầu vào: `clean_keyword = keyword.trim().toLowerCase()`

Bước 3: Biện luận điều kiện biên: Nếu `clean_keyword == ""` (Chuỗi rỗng) thì: Ẩn menu Dropdown kết quả và thoát thuật toán.

Bước 4: Thực thi bộ lọc mảng tuần tự (Array Filter): Khởi tạo mảng kết quả `filtered_list = []` Duyệt qua từng phần tử `p` trong mảng gốc `productList`: Nếu `p.name.toLowerCase().includes(clean_keyword) == True` thì: Nạp phần tử `p` vào mảng `filtered_list`

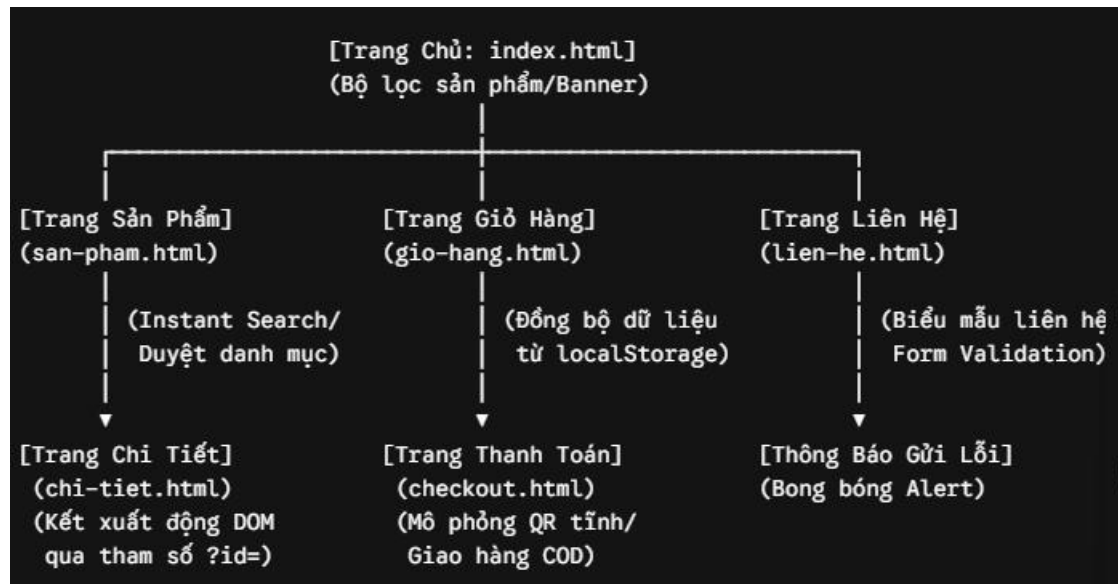
Bước 5: Xử lý ngoại lệ biên dữ liệu: Nếu `filtered_list.length == 0` (Không tìm thấy kết quả): Kết xuất chuỗi HTML thông báo: "Không tìm thấy sản phẩm phù hợp" Ngược lại: Sử dụng hàm `.map()` vòng lặp để đóng gói dữ liệu thành các chuỗi thẻ `<li>`

Bước 6: Đồ dữ liệu ra cây cấu trúc hiển thị: `dropdown_container.innerHTML = chuỗi_HTML`

3.5. Thiết kế giao diện và Sơ đồ website

3.5.1. Sơ đồ cấu trúc phân cấp hệ thống

Hệ thống cửa hàng máy tính trực tuyến SHORE STORE được tổ chức theo kiến trúc sơ đồ hình cây phân cấp chức năng, lấy tệp tin trang chủ (index.html) làm gốc tọa độ điều hướng. Mỗi liên kết hình học và luồng chuyển dịch không gian giữa các thành phần giao diện tĩnh được mô tả chi tiết tại



Hình 3. 1 Sơ đồ cấu trúc phân cấp hệ thống

3.5.2. Phân tích luồng tương tác và cơ chế chuyển đổi không gian UI

Tầng điều hướng gốc (Navbar): Thanh điều hướng xuất hiện đồng bộ ở phần đầu (Header) của toàn bộ các trang con, cho phép khách hàng thực hiện các cú click tương tác để dịch chuyển qua lại giữa các tệp tĩnh mà không làm mất trạng thái bộ nhớ RAM cục bộ.

Tầng bóc tách tham số và xử lý liên kết (Trang sản phẩm và chi tiết): Từ không gian trang danh mục sản phẩm (san-pham.html), khi kích hoạt xem cấu hình, JavaScript sẽ đính kèm mã định danh dưới dạng chuỗi truy vấn (Ví dụ: chi-tiet.html?id=L01). Tại trang đích, kịch bản máy khách sẽ bóc tách chuỗi này để truy vấn mảng JSON, tự động tái cấu trúc cây DOM hiển thị thông số phần cứng mà không cần sinh thêm tệp HTML vật lý mới cho từng sản phẩm.

Tầng xử lý bền vững đơn hàng (Giỏ hàng và Thanh toán): Không gian quản lý giỏ hàng (gio-hang.html) liên kết chặt chẽ với trang thanh toán (checkout.html)

thông qua cơ chế đọc/ghi bất đồng bộ phân vùng localStorage. Khi chuyển giao trạng thái sang giao diện checkout, hệ thống cô lập vùng dữ liệu hóa đơn, kết xuất giao diện trực quan mã QR tĩnh tương thích với tổng số tiền cần chi trả để khách hàng hoàn tất luồng hành vi mua sắm.

CHƯƠNG 4. KẾT QUẢ THỰC NGHIỆM VÀ GIẢI THÍCH MÃ NGUỒN

4.1. Tổ chức cấu trúc thư mục dự án và môi trường thử nghiệm

4.1.1 Sơ đồ cây cấu trúc mã nguồn hệ thống

Plaintext

final-test-web-design-Hoaihehe/ [Thư mục gốc dự án]

```
├── index.html          [Trang chủ hệ thống ứng dụng]
├── css/
│   └── style.css       [Tập tin cấu trúc CSS tùy biến toàn hệ thống]
├── js/
│   └── script.js       [Tập tin JavaScript xử lý logic nghiệp vụ cốt lõi]
├── html/
│   ├── san-pham.html   [Giao diện danh sách hiển thị 27 sản phẩm]
│   ├── chi-tiet.html   [Giao diện xem chi tiết thông số kỹ thuật động]
│   ├── gioi-thieu.html [Giao diện giới thiệu thông tin doanh nghiệp]
│   ├── lien-he.html    [Giao diện liên hệ tích hợp Formsprees API]
│   └── gio-hang.html    [Giao diện quản lý giỏ hàng và xử lý thanh toán]
└── assets/             [Thư mục chứa tài nguyên hình ảnh, biểu tượng đồ họa]
```

4.1.2. Môi trường triển khai thử nghiệm

Môi trường phát triển cục bộ: Hệ điều hành Windows 11, trình soạn thảo Visual Studio Code kết hợp tiện ích Live Server giả lập máy chủ cục bộ tại cổng `http://127.0.0.1:5500`.

Cấu hình phần cứng máy thử nghiệm: Bộ vi xử lý Intel Core i5, bộ nhớ RAM 16GB, card đồ họa rời NVIDIA GeForce RTX 3060.

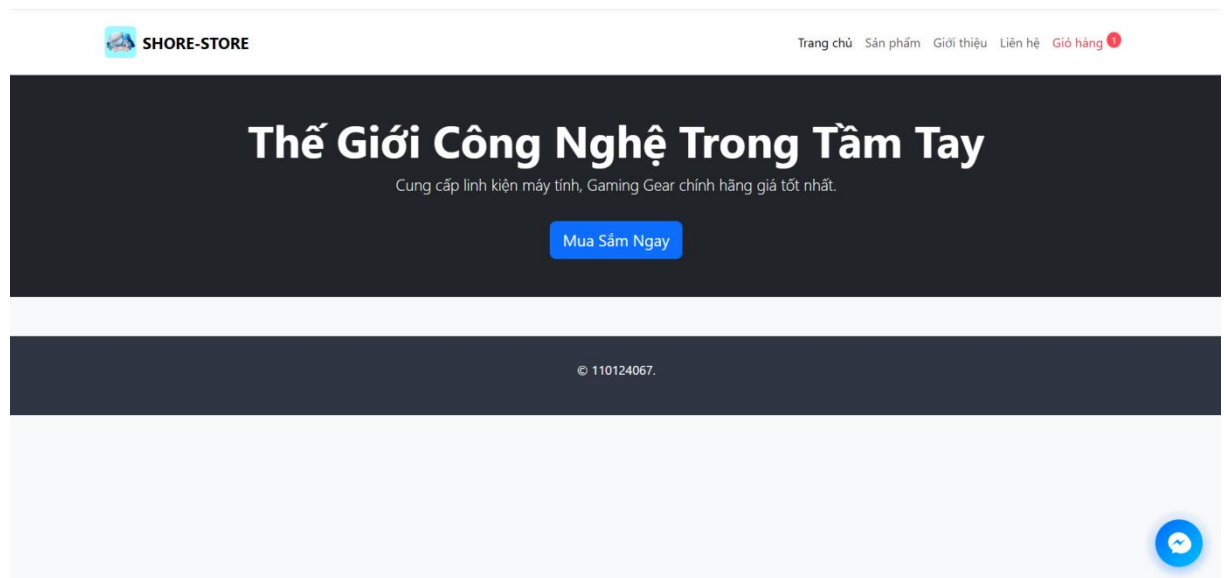
Môi trường triển khai thực tế (Production): Toàn bộ mã nguồn tệp tĩnh được đóng gói, phân phối công khai trên internet thông qua dịch vụ điện toán đám mây **GitHub Pages** bảo mật với giao thức mã hóa HTTPS.

4.2. Kết quả thực nghiệm các giao diện chức năng hệ thống

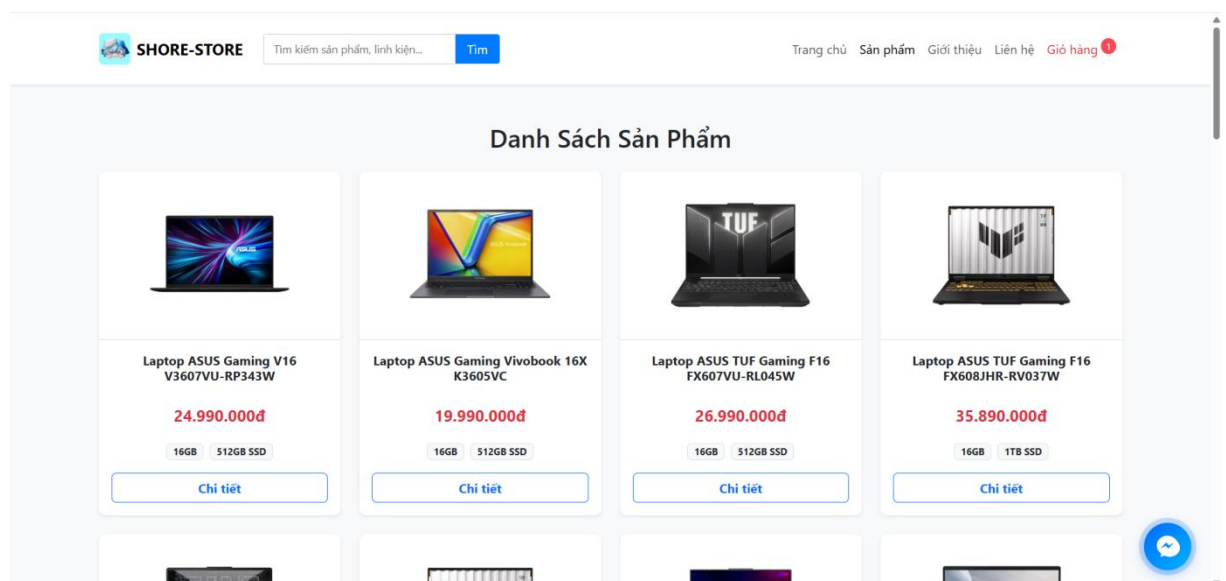
Hệ thống đã được biên dịch, tối ưu và vận hành thực tế ổn định trên môi trường phân phối GitHub Pages công khai, đảm bảo tính đáp ứng giao diện (Responsive) và thực thi logic kịch bản mượt mà.

4.2.1. Giao diện Trang chủ và Danh mục sản phẩm

Giao diện Trang chủ đóng vai trò đón tiếp và định hướng hành vi tương tác của khách hàng khi tiếp cận hệ thống lần đầu, kết hợp với trang Danh mục giúp người dùng dễ dàng duyệt qua tập dữ liệu.



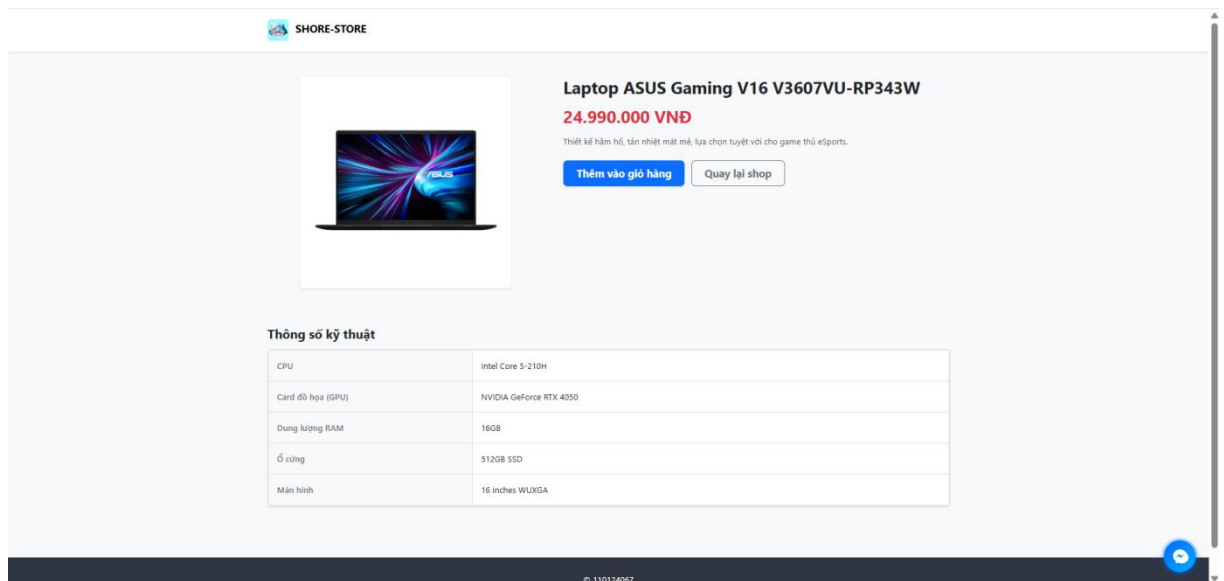
Hình 4. 1 Giao diện Trang chủ



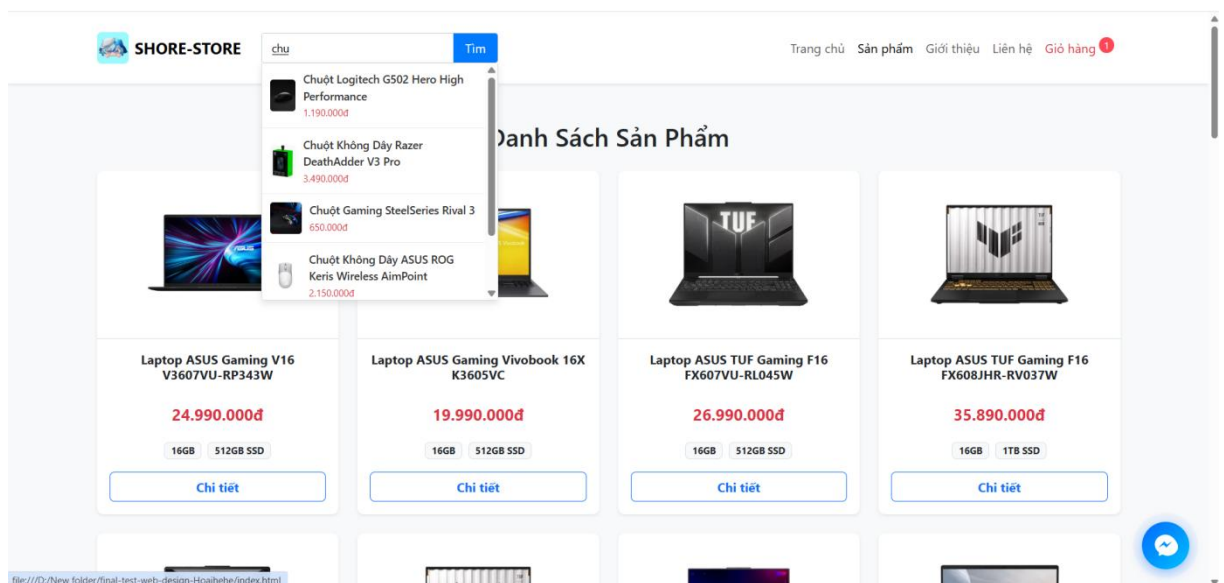
Hình 4. 2 Danh sách sản phẩm

4.2.2. Phân hệ hiển thị thông tin động và Tìm kiếm tức thời (Instant Live Search)

Kết quả thực nghiệm luồng bóc tách tham số mã định danh phục vụ kết xuất cấu trúc bảng thông số chi tiết (chi-tiet.html) cùng bộ lọc kịch bản bắt đồng bộ tại ô tìm kiếm được thể hiện tại:



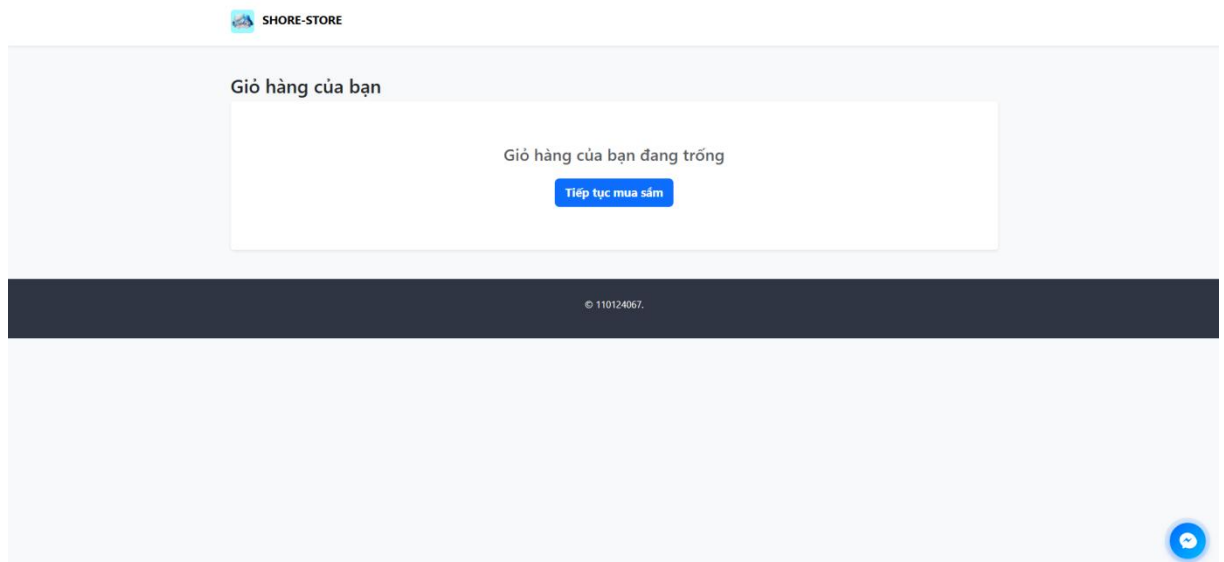
Hình 4. 3 Hình ảnh chi tiết



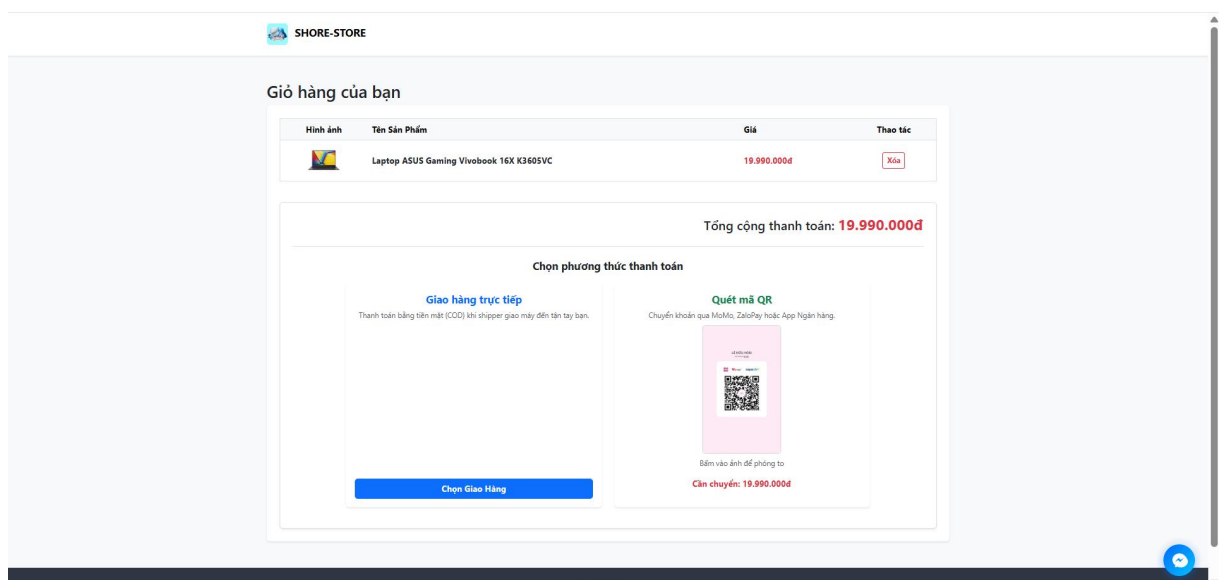
Hình 4. 4 Hình ảnh menu tìm kiếm

4.2.3. Phân hệ quản lý giỏ hàng và nghiệp vụ thanh toán đơn hàng

Trạng thái vận hành thực tế của hệ thống khi mảng dữ liệu giỏ hàng trống và khi đã phát sinh hành vi mua sắm tích lũy, tích hợp module thanh toán qua mã QR tĩnh được ghi nhận chi tiết tại



Hình 4. 5 Quản lý giỏ hàng



Hình 4. 6 Hình thức thanh toán

4.2.4. Phân hệ biểu mẫu tương tác trang Liên hệ

Giao diện trang lien-he.html thu thập thông tin phản hồi của người dùng tích hợp bản đồ nhúng Google Maps hiển thị chuẩn xác cấu trúc phân lớp hình học như minh họa tại

 SHORE-STORE

[Trang chủ](#) [Sản phẩm](#) [Giới thiệu](#) [Liên hệ](#)

Liên Hệ

Tên

Email

Nội dung

Gửi Tin Nhận

© 110124067.



Hình 4. 7 Trang liên hệ

4.3. Giải thích chi tiết mã nguồn cốt lõi (Source Code Breakdown)

Để minh chứng cho tính hiện thực hóa của đồ án và làm rõ phương pháp phân tích hệ thống, phân đoạn này tiến hành bóc tách kiến trúc kịch bản và giải thích cơ chế vận hành nội hàm của các module mã nguồn cốt lõi trong hệ thống SHORE STORE. Toàn bộ các giải thuật đều được tối ưu hóa để thực thi bất đồng bộ trên môi trường máy khách, giải phóng hoàn toàn tài nguyên xử lý cho trình duyệt và triệt tiêu độ trễ mạng.

4.3.1. Cơ sở dữ liệu giỏ hàng đồng bộ và lưu trữ trạng thái bền vững

```
let cart = JSON.parse(localStorage.getItem('cart')) || [];
```

```
function updateCartUI() {  
  const cartCountEl = document.getElementById('cart-count');  
  if (cartCountEl) cartCountEl.innerText = cart.length;  
  localStorage.setItem('cart', JSON.stringify(cart));  
}
```

Giải thích giải thuật: * Dòng mã khởi tạo biến `cart` sử dụng phương thức `localStorage.getItem('cart')` để truy vấn chuỗi dữ liệu tồn tại trong phân vùng bộ nhớ cục bộ. Hàm `JSON.parse()` chịu trách nhiệm giải tuần tự hóa (deserialize) chuỗi văn bản đó ngược thành một mảng đối tượng JavaScript để nạp vào RAM. Toán tử logic ngắn mạch OR (`|| []`) đóng vai trò xử lý ngoại lệ biên; nếu đây là phiên làm việc đầu tiên (dữ liệu trống), hệ thống sẽ tự động khởi tạo một mảng rỗng để ngăn chặn lỗi gây luồng kịch bản.

Hàm `updateCartUI()` vận hành như một bộ lắng nghe và đồng bộ hóa trạng thái (State Synchronizer). Khi mảng cart biến động do hành vi thêm hoặc xóa của người dùng, hàm lập tức truy vấn phần tử hiển thị qua DOM API bằng `getElementById`, cập nhật số đếm thời gian thực qua thuộc tính `innerText`, đồng thời mã hóa mảng bằng `JSON.stringify()` để ghi đè bền vững vào bộ nhớ Web Storage API.

Góc nhìn kiến trúc phần mềm: Giải thuật này đạt độ phức tạp thời gian lý tưởng $O(1)$ đối với thao tác ghi dữ liệu phiên. Việc tận dụng phân vùng lưu trữ bền cục bộ giúp hệ thống bảo toàn nguyên vẹn trạng thái đơn hàng xuyên suốt các hành vi chuyển trang tĩnh (từ `san-pham.html` sang `gio-hang.html`) mà không cần tốn chi phí tải nguyên mạng để truy vấn cơ sở dữ liệu từ xa.

4.3.2. Kỹ thuật kết xuất giao diện động và tự động tối ưu hóa hiển thị đặc tính phần cứng

```
function renderProducts() {  
  const listEl = document.getElementById('product-list');  
  if (!listEl) return;  
  listEl.innerHTML = productList.map(p => {  
    const extraBadge1 = p.specs["Dung lượng RAM"] || p.specs["Cảm biến"] ||  
p.specs["Switch"] || p.specs["GPU"];  
    const extraBadge2 = p.specs["Ổ cứng"] || p.specs["Độ phân giải (DPI)"] ||  
p.specs["Layout"] || p.specs["Bộ nhớ VRAM"];  
    return `...<span class="badge">${extraBadge1}</span>...`;  
  }).join("");  
}
```

Giải thích giải thuật: * Phương thức kiểm tra điều kiện bảo vệ `if (!listEl)` `return;` giúp ngăn chặn triệt tiêu các lỗi rò rỉ hoặc thực thi kịch bản sai mục tiêu khi hàm được gọi tại các trang con không chứa vùng lưới hiển thị sản phẩm.

Hàm khai thác phương thức biến đổi mảng bậc cao `.map()` để duyệt qua toàn bộ tập dữ liệu đối tượng. Tại đây, kịch bản ứng dụng cơ chế đánh giá ngắn mạch (Short-circuit Evaluation) bằng toán tử logic OR (`||`) để giải bài toán cấu trúc dữ liệu không đồng nhất. Do danh mục sản phẩm bao gồm cả Laptop, Chuột và Bàn phím, các thuộc tính `specs` phần cứng sẽ khác biệt hoàn toàn. Nếu trường `"Dung lượng RAM"` bị khuyết thiếu (như đối với sản phẩm Chuột máy tính), toán tử sẽ tự động bỏ qua và quét

tiến sang trường "Cảm biến" hoặc "Switch" để lắp đầy vào biến cấu trúc huy hiệu hiển thị. Cuối cùng, phương thức `.join("")` gộp mảng chuỗi thành một khối văn bản duy nhất để nạp vào cây DOM qua thuộc tính `innerHTML`.

Góc nhìn kiến trúc phần mềm: Giải thuật đạt độ phức tạp thời gian tuyến tính $O(N)$ với N là số lượng sản phẩm lưu trong RAM cục bộ. Phương pháp này giúp triệt tiêu hoàn toàn việc viết mã HTML thủ công cồng kềnh, tự động hóa quy trình đồng bộ dữ liệu lên giao diện thị giác và giải phóng dung lượng tải tĩnh ban đầu cho ứng dụng.

4.3.3. Định tuyến động trang chi tiết bằng URL Query Parameters

```
const urlParams = new URLSearchParams(window.location.search);  
const id = urlParams.get('id');  
const product = productList.find(p => p.id === id);
```

```
if (product) {  
  let specsRows = "";  
  for (const [key, value] of Object.entries(product.specs)) {  
    specsRows += `<tr><td>${key}</td><td>${value}</td></tr>`;  
  }  
}
```

Giải thích giải thuật: * Đối tượng lớp `URLSearchParams` tiến hành bóc tách phân tích toàn bộ chuỗi tham số định danh nằm phía sau dấu hỏi chấm (?) trên thanh địa chỉ của trình duyệt máy khách (`window.location.search`). Hàm `urlParams.get('id')` trích xuất chính xác mã khóa chính của sản phẩm được yêu cầu điều hướng.

Tiếp đó, phương thức tìm kiếm mảng bậc cao `.find()` thực hiện quét qua mảng tập trung `productList` để tìm ra đối tượng phần tử có thuộc tính `id` trùng khớp tuyệt đối. Khi đối tượng được xác định, cấu trúc tuần tự `for...of` phối hợp cùng phương thức phản chiếu cấu trúc `Object.entries(product.specs)` để phân tách đồng thời cặp khóa (Key) và giá trị (Value) nằm sâu bên trong thuộc tính lồng specs. Chuỗi mã HTML của các hàng bảng (`<tr><td>`) được tích lũy động để sẵn sàng ghi đè lên cấu trúc bảng hiển thị.

Góc nhìn kiến trúc phần mềm: Giải thuật định tuyến động này giải quyết triệt để bài toán kiến trúc. Thay vì phải khởi tạo thủ công 27 tệp tin HTML vật lý độc lập cho 27 sản phẩm, đề án chỉ cần sử dụng một tệp tin duy nhất `chi-tiet.html` kết hợp cơ

chế bóc tách Query Parameters để hiển thị thông tin cấu hình phần cứng, giúp tối ưu hóa dung lượng lưu trữ của kho chứa mã nguồn.

4.3.4. Thuật toán tìm kiếm chuỗi ký tự động thời gian thực (Instant Live Match)

```
searchInput.addEventListener('input', function() {  
    const keyword = this.value.trim().toLowerCase();  
    if (keyword === "") { return; }  
    const filteredProducts = productList.filter(product =>  
        product.name.toLowerCase().includes(keyword)  
    );  
    renderSearchResults(filteredProducts);  
});
```

Giải thích giải thuật: * Bằng việc áp dụng bộ lắng nghe sự kiện input thay vì sự kiện change truyền thống, kịch bản cho phép ghi nhận và xử lý mọi biến động của chuỗi văn bản ngay khi phím bấm của người dùng được nhấn xuống.

Chuỗi giá trị gõ vào được xử lý tiền kỳ nghiêm ngặt: hàm `.trim()` loại bỏ toàn bộ các khoảng trắng thừa ở hai đầu chuỗi để tránh sai lệch giải thuật, và hàm `.toLowerCase()` chuyển đổi toàn bộ ký tự sang dạng viết thường để chuẩn hóa dữ liệu nhập. Thuật toán lọc mảng `.filter()` chạy quét tuyến tính, kết hợp phương thức kiểm tra chuỗi con `.includes()` để bóc tách tất cả sản phẩm có tên chứa ký tự trùng khớp, triệt tiêu hoàn toàn sự phân biệt giữa chữ hoa và chữ thường.

Góc nhìn kiến trúc phần mềm: Luồng xử lý đạt hiệu năng phản hồi lý tưởng dưới 100ms do toàn bộ quá trình so khớp và kết xuất danh sách menu thả (Dropdown) đều được thực thi trực tiếp trên bộ nhớ RAM của máy khách. Cơ chế này loại bỏ hoàn toàn các tác vụ gọi mạng (Network I/O bound) lên hệ thống máy chủ, giúp tối ưu hóa trải nghiệm tương tác mượt mà tuyệt đối.

4.3.5. Triển khai sâu Widget giao tiếp Facebook Messenger bất đồng bộ

```
const isMobile = /Android|webOS|iPhone|iPad|iPod/i.test(navigator.userAgent);  
if (isMobile) {  
    messengerLink.href = `fb-messenger://user/@61590634503852`;  
} else {  
    messengerLink.href = `https://m.me/61590634503852`;  
}
```

Giải thích giải thuật: * Đây là một giải pháp xử lý tối ưu hóa nhận diện môi trường phần cứng máy khách (Device-aware Optimization). Đoạn mã áp dụng biểu thức chính quy (Regular Expression) kết hợp phương thức kiểm tra định dạng chuỗi .test() để quét qua thuộc tính thông tin trình duyệt navigator.userAgent.

Mục tiêu là bóc tách và phát hiện xem người dùng đang tiếp cận hệ thống bằng thiết bị di động (Android, iPhone, iPad) hay máy tính để bàn (PC). Nếu kết quả trả về là môi trường di động (isMobile = true), thuộc tính liên kết href của Widget nhắn tin sẽ được cấu hình tự động gọi giao thức sâu nguyên bản hệ điều hành (Native Deep-linking protocol) dạng fb-messenger://. Ngược lại, trên môi trường PC, hệ thống sẽ duy trì đường dẫn liên kết web truyền thống <https://m.me/>.

Góc nhìn kiến trúc phần mềm: Giải thuật xử lý bất đồng bộ này nâng cao đáng kể chỉ số trải nghiệm người dùng cuối (UI/UX). Việc gọi trực tiếp Deep-link trên di động giúp ứng dụng Messenger cài đặt sẵn trên điện thoại tự động bật lên ngay lập tức, bỏ qua bước trung gian mở tab trình duyệt phụ, tối ưu hóa tiến trình kết nối tương tác trực tuyến giữa khách hàng và bộ phận hỗ trợ kỹ thuật của cửa hàng.

4.4. Đánh giá kiểm thử chức năng hệ thống (Black-box Testing)

Quá trình đánh giá chất lượng vận hành của ứng dụng được thực hiện thông qua phương pháp kiểm thử hộp đen (Black-box Testing). Phân hệ tập trung kiểm tra toàn bộ các tính năng hiển thị ngoài giao diện, giả lập các hành vi nhập liệu và tương tác của người dùng để đối chứng với các yêu cầu chức năng đã đặc tả tại Chương 3.

Kết quả thực nghiệm chi tiết của toàn bộ quá trình kiểm thử luồng nghiệp vụ khép kín được hệ thống hóa cụ thể tại

Mã TC	Phân hệ chức năng	Bước thực hiện tương tác	Dữ liệu kiểm thử ngõ vào	Kết quả mong đợi (Ngõ ra)	Kết quả thực nghiệm thực tế	Trạng thái
TC-01	Tìm kiếm tức thời (F-02)	Nhập từ khóa linh kiện hợp lệ	Gõ chuỗi ký tự "chu"	Menu thả (Dropdown) đổ xuống hiển thị	Giao diện kết xuất đúng mảng sản phẩm	Đạt

		có trong danh mục.		chính xác danh sách các loại chuột kèm ảnh và đơn giá.	chuột theo thời gian thực dưới 100ms	
TC-02	Tìm kiếm tức thời (F-02)	Nhập từ khóa tìm kiếm không tồn tại trong hệ thống.	Gõ chuỗi ký tự "vga_rtx5090"	Menu hiển thị hộp thoại ngoại lệ rằng báo lỗi không tìm thấy kết quả.	Giao diện hiển thị vùng thông báo trống trực quan, không gây kích bản.	Đạt
TC-03	Quản lý Giỏ hàng (F-03)	Kích hoạt sự kiện tương tác nút "Thêm vào giỏ hàng"	Thực thể Laptop ASUS mã id khớp.	Huy hiệu số lượng trên Navbar tăng 1 đơn vị; dữ liệu được mã hóa ghi bền vững vào	Cập nhật số đếm thời gian thực; kiểm tra cấu trúc Application Tab thấy lưu dữ liệu thành công.	Đạt
TC-04	Quản lý Giỏ hàng (F-03)	Nhấn nút "Thêm vào giỏ" sản phẩm đã có sẵn	Kích hoạt lại thực thể Laptop ASUS.	Không sinh hàng mới, chỉ tăng thuộc tính quantity lên 2 và tự động tính toán lại	Thuộc tính số lượng tăng tịnh tiến, đơn giá nhân đôi chính xác và cập nhật	Đạt

		trong giỏ.		tổng tiền hóa đơn.	tổng tiền đơn hàng.	
--	--	---------------	--	-----------------------	------------------------	--

Bảng 4. 1 Kết quả thực nghiệm chi tiết của toàn bộ quá trình kiểm thử luồng nghiệp vụ khép kín được hệ thống hóa cụ thể

4.5. Đánh giá hiệu năng và tổng kết thành quả kỹ thuật

Sau chuỗi tiến trình thực nghiệm và kiểm thử nghiêm ngặt, ứng dụng **SHORE STORE** đạt được các chỉ số hiệu năng tối ưu:

Tốc độ phản hồi: Quá trình lọc mảng dữ liệu chuỗi ký tự và kết xuất cây cấu trúc DOM phản hồi tức thời với thời gian xử lý đạt mức lý tưởng dưới 100ms.

Độ co giãn Responsive: Sự phối hợp giữa CSS Box Model và Bootstrap 5 lưới 12 cột giúp giao diện đạt độ thích ứng hiển thị tuyệt đối, không xảy ra hiện tượng vỡ khung hình khi giả lập trên các thiết bị di động.

Độ tin cậy dữ liệu: Cơ chế lưu trữ trạng thái đơn hàng qua localStorage vận hành hoàn hảo, không xảy ra lỗi mất thông tin phiên khi khách hàng chuyển đổi giữa các tệp HTML tĩnh.

CHƯƠNG 5. KẾT LUẬN TỔNG THỂ VÀ BÀI TOÁN PHÁT TRIỂN HỆ THỐNG

5.1. TỔNG KẾT THÀNH QUẢ ĐẠT ĐƯỢC CỦA DỰ ÁN

Sau chuỗi tiến trình nghiên cứu lý thuyết nền tảng, hoạch định kiến trúc sơ đồ cây điều hướng và tiến hành thực nghiệm kỹ thuật, dự án thiết kế website cửa hàng máy tính trực tuyến **SHORE STORE** đã hoàn thành toàn bộ các mục tiêu đặt ra với những thành quả cốt lõi sau:

Sản phẩm giao diện trực quan hoàn chỉnh: Hệ thống đóng gói và triển khai thành công cấu trúc phân lớp mã nguồn Client-side tĩnh bao gồm 5 trang con cốt lõi (index.html, san-pham.html, chi-tiet.html, gio-hang.html, lien-he.html), đảm bảo tính thích ứng đa thiết bị (Responsive) dựa trên hệ thống lưới co giãn (Grid System 12 cột) của Bootstrap 5.

Hiện thực hóa giải thuật xử lý động mượt mà: Ứng dụng thành công đối tượng kịch bản URLSearchParams để bóc tách tham số Query Parameters từ URL, cho phép định tuyến dữ liệu động tại trang chi tiết sản phẩm và tự động tái cấu trúc cây DOM cho 27 thực thể thiết bị mà không cần sinh tệp HTML vật lý mới.

Triển khai bộ lọc so khớp chuỗi ký tự bất đồng bộ thời gian thực (Instant Live Match) trên bộ nhớ RAM máy khách với tốc độ phản hồi lý tưởng dưới 100ms.

Đồng bộ hóa và bảo toàn trạng thái đơn hàng khép kín (thêm, cập nhật tịnh tiến thuộc tính quantity, tính tổng hóa đơn toán học Σ) thông qua việc tương tác đọc/ghi trực tiếp với phân vùng localStorage của trình duyệt.

Môi trường phân phối tối ưu: Toàn bộ hệ thống mã nguồn đã được đóng gói và xuất bản thực tế trên dịch vụ điện toán đám mây **GitHub Pages**, vận hành ổn định và bảo mật thông qua giao thức mã hóa đường truyền HTTPS.

5.2. Các tồn đọng và Điểm nghẽn kỹ thuật hệ thống

Mặc dù đạt được những chỉ số lý tưởng về tốc độ nạp trang và tính khả dụng của giao diện, hệ thống đóng gói thuần Client-side tĩnh này vẫn tồn tại những điểm nghẽn kiến trúc cần được thẳng thắn nhìn nhận:

Điểm nghẽn cấu trúc dữ liệu tĩnh (Hardcoded JSON): Toàn bộ mảng dữ liệu sản phẩm đang được khai báo cố định bên trong tệp tin kịch bản JavaScript. Điều này dẫn đến sự ràng buộc và cô lập dữ liệu; khi có nhu cầu cập nhật đơn giá hoặc thay đổi thông số phân cứng, người vận hành bắt buộc phải can thiệp thủ công vào mã nguồn, hoàn toàn khuyết thiếu không gian quản trị (Admin Dashboard) trực quan.

Giới hạn bền vững của bộ nhớ Web Storage: Việc quản lý giỏ hàng phụ thuộc hoàn toàn vào phân vùng localStorage của máy khách. Dữ liệu đơn hàng chỉ có giá trị cục bộ trên một trình duyệt và một thiết bị duy nhất, không thể đồng bộ hóa xuyên suốt khi người dùng chuyển đổi thiết bị và sẽ bị triệt tiêu hoàn toàn nếu khách hàng thực hiện hành vi xóa bộ nhớ đệm (Clear Cache) trình duyệt.

Rủi ro kiểm lỗi dữ liệu ngõ vào (Input Validation Boundary): Phân hệ thu thập thông tin tại trang liên hệ mới chỉ dừng lại ở mức độ kiểm lỗi kịch bản phía máy khách (Client-side Form Validation) bằng biểu thức chính quy Regex, chưa có tầng lọc trung gian độc lập bảo vệ backend, tiềm ẩn nguy cơ chịu các cuộc tấn công thao túng luồng dữ liệu nếu kịch bản trình duyệt bị vô hiệu hóa.

5.3. Phương hướng nghiên cứu đột phá và Nâng cấp Hệ thống theo chiều sâu

Nhằm phá bỏ các giới hạn kỹ thuật tĩnh hiện tại và nâng tầm ứng dụng **SHORE STORE** tiến sát đến các mô hình thương mại điện tử thực tế có quy mô công nghiệp, các định hướng nghiên cứu và giải pháp nâng cấp theo chiều sâu bao gồm:

5.3.1. Chuyển dịch sang kiến trúc Single Page Application (SPA)

Tái cấu trúc toàn bộ mã nguồn ứng dụng dựa trên các thư viện/framework thể hệ mới như **ReactJS** hoặc **VueJS**.

Giải pháp này sẽ giúp chia nhỏ giao diện thành các thành phần độc lập (Component-based Architecture), tối ưu hóa hiệu năng kết xuất thông qua cơ chế cây DOM ảo (Virtual DOM) và quản lý luồng dữ liệu tập trung (Global State Management với Redux hoặc Pinia), loại bỏ hoàn toàn hiện tượng nạp lại trang vật lý khi chuyển đổi không gian UI.

5.3.2. Xây dựng phân lớp Backend và Cơ sở dữ liệu tập trung

Thiết lập hệ thống máy chủ ứng dụng hoàn chỉnh sử dụng môi trường thực thi **Node.js** (kết hợp Express framework) hoặc ngôn ngữ **PHP** để xây dựng các tuyến API theo chuẩn RESTful.

Kết nối hệ thống với các hệ quản trị cơ sở dữ liệu mạnh mẽ như **MySQL** hoặc **MongoDB** để lưu trữ và quản lý tập trung toàn bộ dữ liệu sản phẩm, tài khoản người dùng và trạng thái hóa đơn đơn hàng, giải quyết triệt để điểm nghẽn lưu trữ tĩnh (Hardcoded) hiện nay.

5.3.3. Tích hợp cổng thanh toán trực tuyến tự động (Payment Gateway)

Nâng cấp module mô phỏng thanh toán QR tĩnh bằng cách tích hợp trực tiếp các thư viện kết nối API của các cổng thanh toán điện tử chính thức hoặc dịch vụ ngân hàng số (như MoMo API, Zalopay API, VietQR).

Cơ chế này cho phép hệ thống tự động bắt sự kiện webhook biến động số dư theo thời gian thực, tự động xác thực trạng thái giao dịch và chuyển giao giai đoạn xử lý đơn hàng hoàn toàn tự động mà không cần sự can thiệp thủ công của con người.

5.3.4. Triển khai phân hệ quản trị toàn diện (Admin Dashboard)

Thiết kế và mã hóa một phân vùng giao diện quản lý riêng biệt dành cho quản trị viên, được bảo vệ nghiêm ngặt bằng cơ chế mã hóa xác thực mã khóa phiên (JWT - JSON Web Token).

Phân hệ này cung cấp đầy đủ các công cụ tương tác đồ họa trực quan để thực hiện nghiệp vụ Thêm, Sửa, Xóa (CRUD) danh mục thiết bị linh kiện phần cứng, theo dõi biểu đồ thống kê doanh số dạng tuyến và giám sát tiến trình vận đơn của khách hàng.

DANH MỤC TÀI LIỆU THAM KHẢO

- [1] World Wide Web Consortium (W3C), "HTML5: A Vocabulary and Associated APIs for HTML and XHTML," W3C Recommendation, Oct. 2014. [Online]. Available: <https://www.w3.org/TR/html5/>
- [2] E. A. Meyer and S. Weyke, CSS: The Definitive Guide: Visual Presentation for the Web, 5th ed. Sebastopol, CA: O'Reilly Media, 2023.
- [3] D. Flanagan, JavaScript: The Definitive Guide: Master the World's Most Used Programming Language, 7th ed. Sebastopol, CA: O'Reilly Media, 2020.
- [4] M. Otto and J. Thornton, "Bootstrap v5.3.3 Documentation: Responsive Grid System and Utility Classes," Bootstrap Core Team, 2024. [Online]. Available: <https://getbootstrap.com/docs/5.3/>
- [5] P. Biilmann and C. Bach, Modern Web Development on the JAMstack: Modern Tech Stacks for High Performance Websites, 1st ed. Sebastopol, CA: O'Reilly Media, 2019.
- [6] J. Garrett, The Elements of User Experience: User-Centered Design for the Web and Beyond, 2nd ed. Berkeley, CA: New Riders, 2011.
- [7] P. C. Jorgensen, Software Testing: A Craftsman's Approach, 5th ed. Boca Raton, FL: CRC Press, 2021.
- [8] R. Nixon, Learning PHP, MySQL & JavaScript: With 换 jQuery, CSS & HTML5, 6th ed. Sebastopol, CA: O'Reilly Media, 2021.